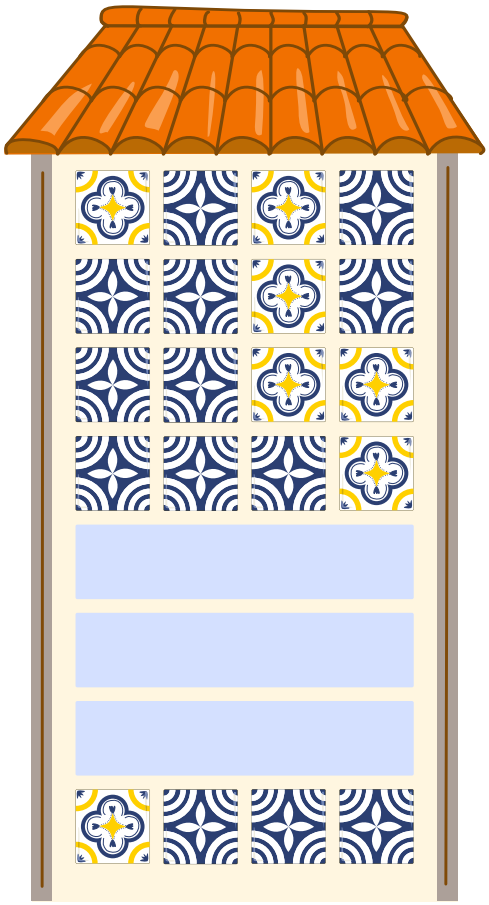
Contents

1 Ajulejos

A house wall is to be covered with azulejos (Portuguese tiles). The pattern consists of yellow tiles  and blue tiles  . These are arranged in such a way that exactly one tile must change from row to row.



Three rows are missing from the pattern. Which option shows the correct second missing row?

A. 

B. 

C. 

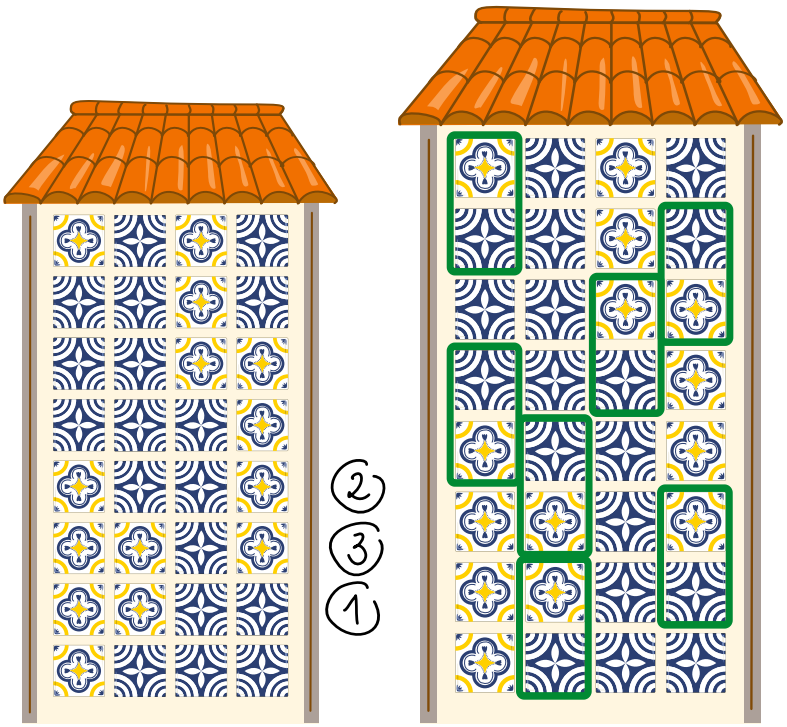
D. 

Solution

C is the correct option.

Since only one tile must be changed from row to row, only this sequence can be chosen in this task. We have added 2 more rows after the missing rows to cover the wall.

In the following graphic, the green borders mark the areas of the respective rows that differ in one place.



There is no other combination of the rows that meet the specifications.

This is Computer Science!

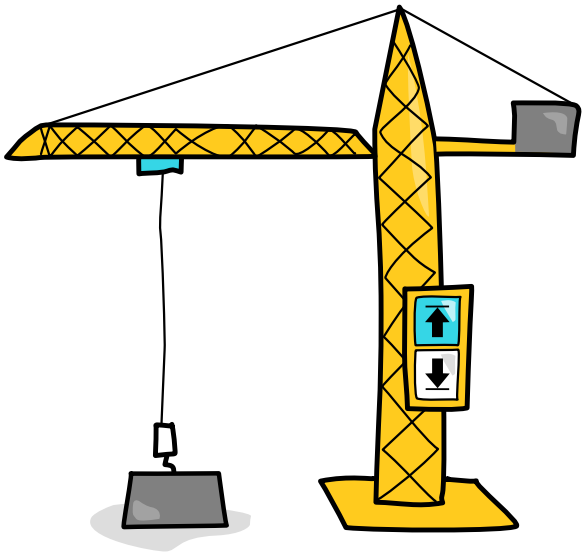
The Gray code is a special form of coding in which adjacent code words always differ in exactly one position. The difference between two code words is therefore always 1 (also known as the Hamming distance). This property remains constant between all consecutive code words. In addition, the code is cyclic, which means that the last and first code words also differ in only one position.

This property helps to reduce transmission errors, as it prevents incorrect intermediate states from occurring when several positions are changed simultaneously, for example due to signal interference or signal distortion. This is why Gray code is one of the most robust, error-avoiding coding methods.

Websites And Keywords

–

2 Toy Crane



This toy crane has two buttons that move the box either up  or down  . At the start of the day, the box is at the bottom. The box moves only when it is possible to do so. For example, if the box is already at the top, pressing  again does not move it any higher. Amit pressed the buttons randomly:

How many times did the box move upwards?

A) 4

B) 5

C) 8

D) 10

Solution

The correct answer is (B).

The box moved up a total of five times during the day.

The box is initially down, hence the first time the button  is pressed, the box moves up.

From then on, if someone presses that button when the box is already up, the box does not move.

The box will move up whenever the sequence shows this pattern: 

Its occurrences are shown here:

If you chose the number 4, you probably counted the pattern, but forgot to include the first move.

If you chose the number 8, you probably counted how many times someone pressed  button, even if the box did not move.

If you chose the number 10, you probably counted how many times the box moved up but also when it moved down.

This is Computer Science!

This task is about processing encoded information. The sequence of arrows ↓ (down) and ↑ (up) is a compact code describing button presses, and it can be analysed directly to obtain new information: how many times the crane was raised.

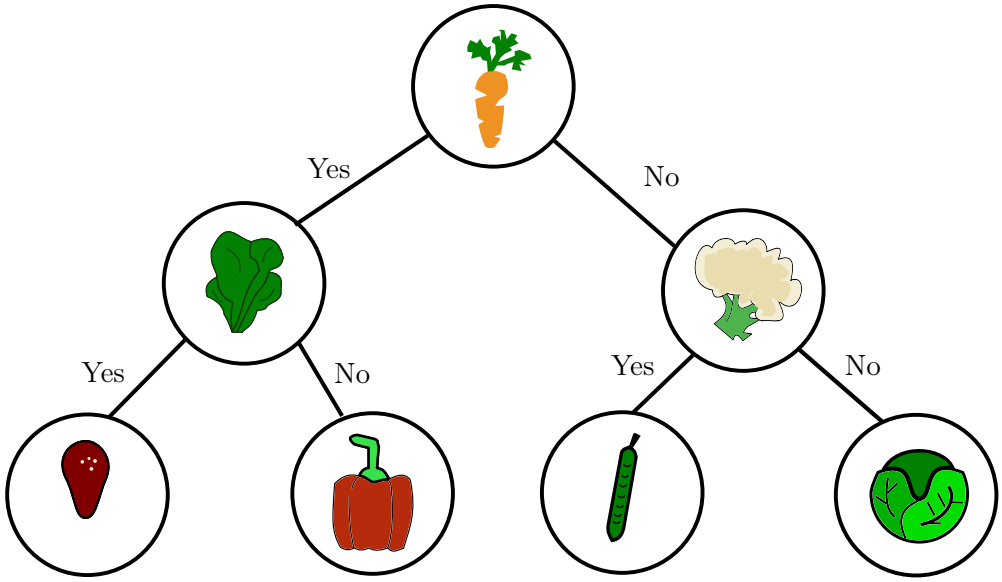
The whole sequence does not need to be decoded into a complete story of the day. Instead, it is enough to look for meaningful changes in the code. In this case, a raising event happens when the state changes from ↓ to ↑. This illustrates an important informatics idea: encoded data can often be processed using simple rules to detect events, count occurrences, or extract useful information without reconstructing every detail of the original situation.

Websites And Keywords

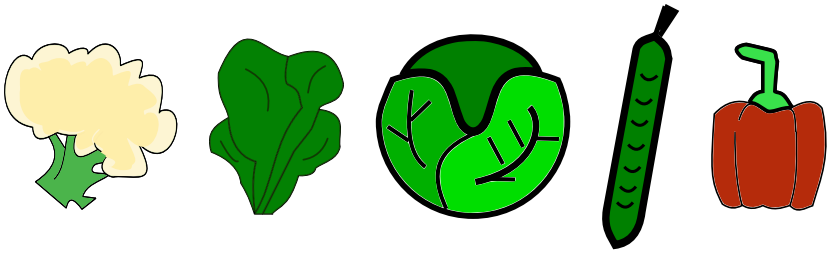
–

3 Picky Eater

Mika uses the following paths to decide what to eat for dinner. She wants to end up with Cutlet  . Mika starts with the carrot  at the top. At each food item, she asks herself, "Should I eat this?". If the answer is Yes, she goes left. If the answer is no, she goes right. She continues until she reaches the Cutlet at the bottom-left corner.



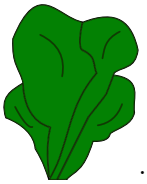
Which food item besides the carrot does Mika have to eat to finally reach the Cutlet?



A) B) C) D) E)

Solution

The correct answer is B.

Look at the path and find the cutlet  on the left corner. To get there from the top, Mika must follow the «Yes» arrow at the carrot  and then the «Yes» arrow at the spinach  . So, the second vegetable she eats is spinach.

This is Computer Science!

Have you ever felt confused when making a choice? A tool called a Decision Tree can help. You used one in this task. Even though the picture in this task resembles a "tree", it does not look much like a real tree in nature. In Informatics, trees are drawn with the root at the top. We move along the path lines by answering one question at a time until we reach the final answer.

In this task, Mika used the decision tree to find the path to the food item she likes. Instead of asking, «What do I want?», she followed the rules: «If I say Yes to carrots, where does the path take me?» This logical step helped her reach the cutlet.

Computers use decision trees too! In Informatics, they help computers recognize objects in pictures and even recommend videos you might like. Just like Mika followed the path lines to get her dinner, computers follow decision trees to solve problems and give useful results.

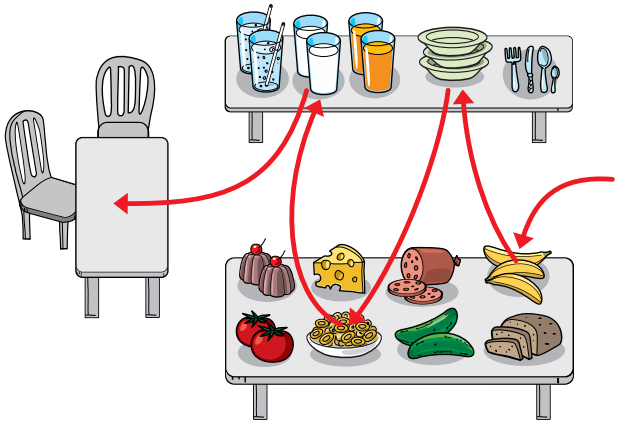
Websites And Keywords

TODO

4 Breakfast

These are two breakfast tables.

Marko walked to get things as the red path shows.



Which of the following statements is incorrect?

A) Marko got milk last.

B) After the bowl, Marko got chakli.

C) Marko did not get tomato.

D) First, Marko got a bowl.

Solution

Correct answer is D) First, Marko got a bowl.

Marko came into the room from the right side of the picture.

He got bananas, a bowl, chakli and milk, in this exact order.

A) is true (milk was last)

B) is true (arrow points from bowls to chakli)

C) is true (no arrow to tomato)

D) is false (first item was a banana)

This is Computer Science!

In this task, information is shown using a picture and a red path with arrows. The picture represents a real-life situation, and the red path shows the order of actions. To solve the task, you need to carefully follow the path and arrange the steps in the correct order.

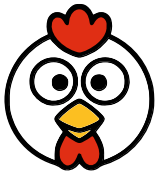
In Informatics, putting actions in the right sequence is very important. Computers and programs must follow instructions step by step to work correctly. This task helps practice understanding information, following sequences, and organizing actions in the correct order.

Websites And Keywords

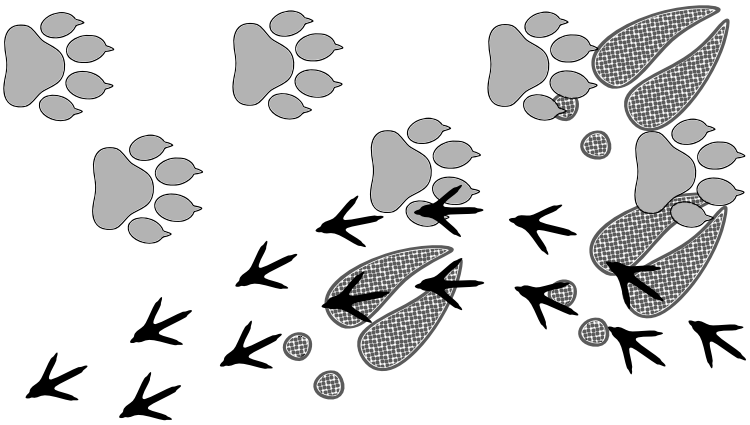
TODO

5 The Mysterious Footprints

Bunny was walking in the forest after a light snowfall. He noticed some animal footprints in the snow. The animals whose footprints he saw are listed below.

Animal	Footprints
 dog	
 deer	
 chicken	

The footprints he saw looked like this:



In what order did the animals walk through the forest?

A) Dog → Deer → Chicken

B) Deer → Dog → Chicken

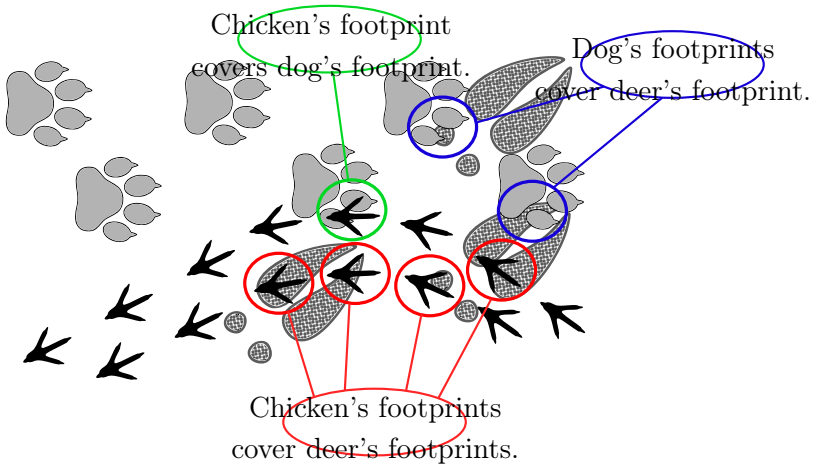
C) Chicken → Dog → Deer

D) Deer → Chicken → Dog

Solution

B is the correct option.

If one animal's footprints are covered by another animal's footprints, then the animal whose footprints are covered passed earlier, and the animal whose footprints are on top passed later.



Next, we focus on the areas where the footprints overlap.

- The deer's hoof prints are covered by both the dog's paw prints and the chicken's tracks. This means the deer passed before both the dog and the chicken. - The dog's paw prints are on top of the deer's prints, but they are covered by the chicken's tracks. So the dog passed after the deer but before the chicken.
- The chicken's tracks are on top of all the others and are the clearest. This means the chicken passed last.

This is Computer Science!

This task models a classic informatics problem of extracting structure from observational data. The overlapping footprints in the snow can be interpreted as a set of pairwise precedence constraints, where each overlap encodes which animal passed before another. From these local comparisons, learners implicitly construct a directed graph in which nodes represent animals and edges represent time relations. The challenge is therefore equivalent to identifying a consistent global ordering that satisfies all constraints, just as in well-known problems such as task scheduling, dependency resolution in software systems, or ordering instructions in computation.

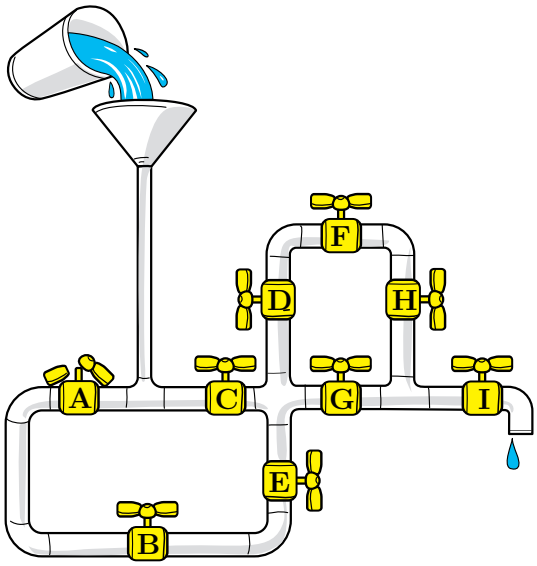
From a more formal perspective, the situation corresponds to working with a partial order that must be extended into a linear order (a total sequence). Solving the task mirrors the logic behind topological sorting algorithms, where only certain elements can be directly compared and multiple valid solutions may exist if constraints are not complete. In this way, the visual reasoning about footprint overlaps becomes an introduction to key informatics ideas such as graph representations, constraint satisfaction, and algorithmic ordering, demonstrating how real-world observations can be transformed into structured data and systematically processed to obtain a valid solution.

Websites And Keywords

–

6 Stop the Water

Bunty is playing with pipes and pouring water into them. Along the pipes, there are taps labelled A to I that can be opened and closed to control the flow of water. But tap A is broken and cannot be closed.



Which taps should be closed to prevent water from reaching tap E, using the fewest possible taps?

A) Taps A and C

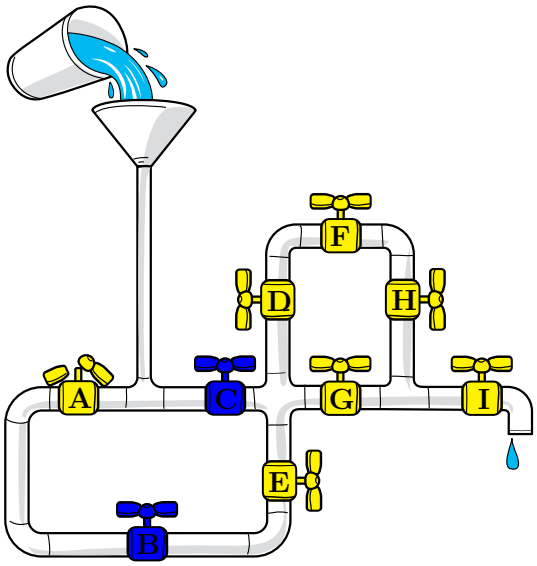
B) Taps C and F

C) Taps B, C and F

D) Taps B and C

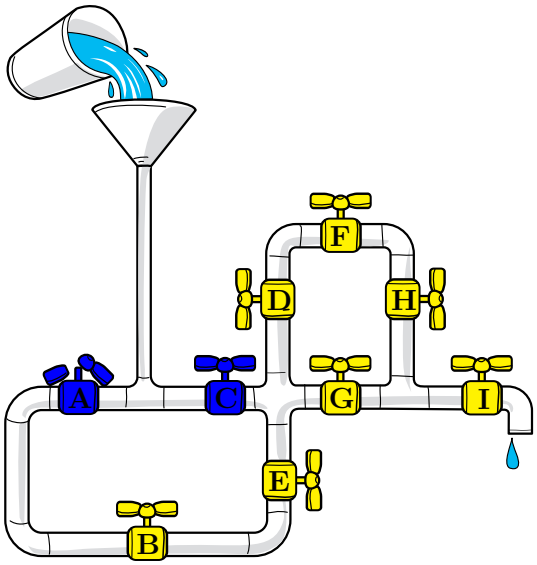
Solution

The correct answer is (D).



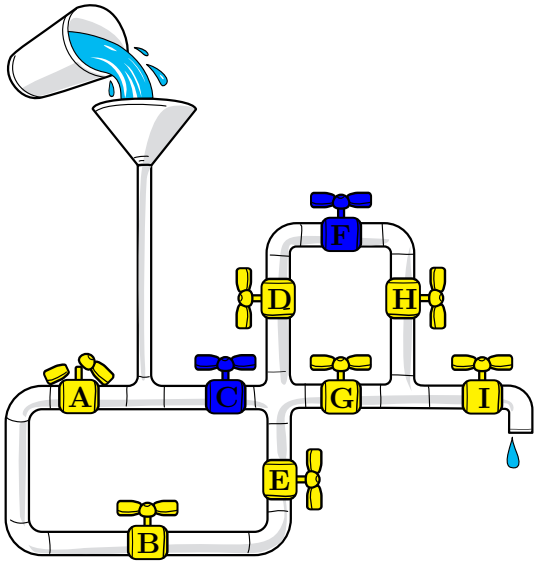
Two paths lead the water to Tap E: A -> B -> E and C -> E. Since Tap A is broken, then closing B and C blocks the path to E using the least amount of taps.

Answer A is incorrect



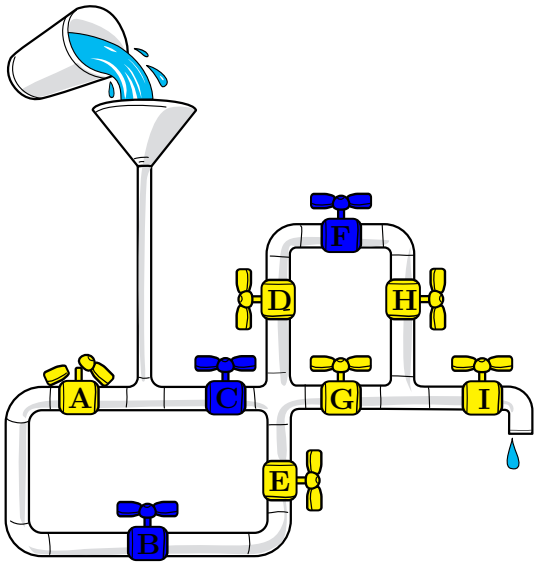
Closing A and C would block both paths to E but the question states that tap A is broken.

Answer B is incorrect



Closing C blocks one path, but water still reaches E by traveling A -> B -> E.

Answer C is incorrect



Closing B and C does block both paths to E, but F is unnecessary.

This is Computer Science!

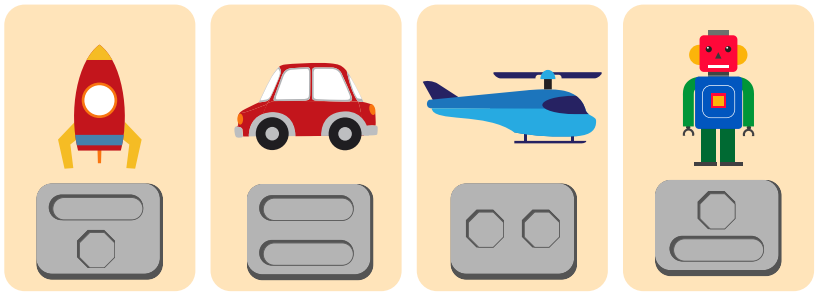
In this task, you explored how water flows between connected places. The connected pipes form a network. Computers use networks to represent many systems in real life, like how cars travel on roads, how water gets to your house and how information travels across the internet. By closing taps, you changed the paths the water could take. This is similar to how computers systems control how information travels across networks, for example, to stop viruses from spreading or to guide messages to the right person.

Websites And Keywords

–

7 Toys and Batteries

Milu has some toys that need batteries to work. The shapes of batteries needed for each toy are shown in the picture below.



Milu's mother found some batteries at home that can be used in these toys. All of these batteries shown in the picture below can only be used once.

Milu wants to power as many toys as possible at the same time

What is the greatest number of toys that can be powered at the same time using these batteries?

A) 1

B) 2

C) 3

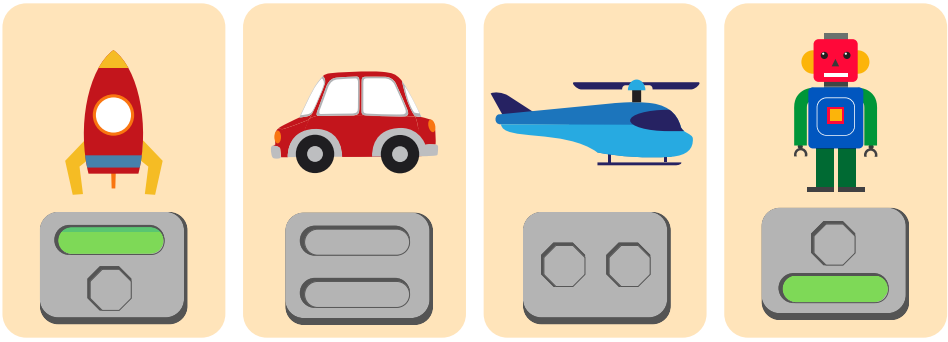
D) 4

Solution

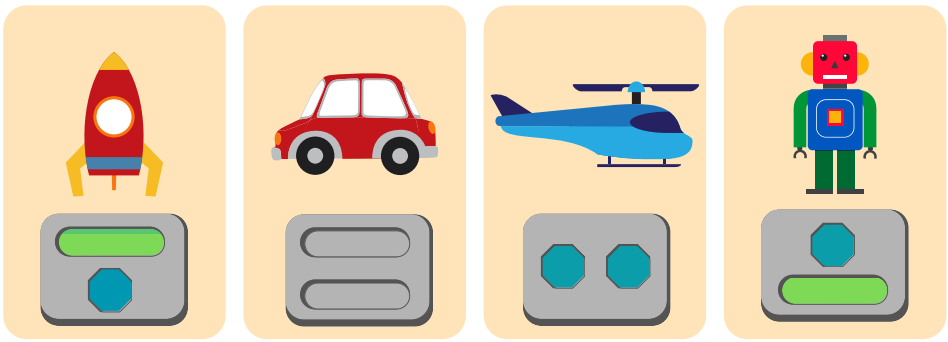
The correct answer is (C).

It is not possible to power all four toys at the same time because it would require four long batteries, and we only have two.

Knowing this, we start by trying to use the two long batteries. One option is as shown:



Then, when we try to use the four remaining batteries, we see that it is possible to power three toys as shown:



This shows us it is possible to power three toys at the same time and so that must be the answer.

(Note that if we had used our two long batteries to power the car, we would only have been able to power a total of two toys. Also, it is not helpful to only use one long battery for the car.)

This is Computer Science!

Each toy is powered only if matching shape batteries are used as needed for that toy. These are constraints that we must satisfy. Among the ways to satisfy these constraints, we want the way that powers as many toys as possible. We call this the optimal solution.

Many real-world situations involve trying to find optimal solutions to constraint satisfaction problems. You can imagine complex systems such as airport schedulers, factory assembly lines and balancing financial portfolios.

A computer scientist's role in these real-world situations is often to find an optimization algorithm that finds an optimal solution satisfying the given constraints. For this task, we considered the maximum theoretical answer (powering four toys), determined it was not possible, and then found a way to use batteries in the next best possible scenario (powering three toys). It is somewhat unusual for this kind of approach to work in the real-world.

Another way to solve the task would be to consider all possible ways to use the batteries. This is called an exhaustive search. This tends to work more often in the real-world but can sometimes be too slow. When that happens, computer scientists go to their problem-solving tool-box in search of more sophisticated algorithms.

Websites And Keywords

TODO

8 Amelia's Message

Amelia sends messages to her friends using animal stickers  .

At the end of each message, Amelia adds 1 or more special stickers as follows:

- If the message contains exactly two animals, she adds a star:
- If the message contains exactly three animals, Amelia adds a heart: 
- If all the animals in the message are all different from each other, Amelia adds a sun:

Which of the following messages could Amelia have sent??

A) 

B) 

C) 

D) 

Solution

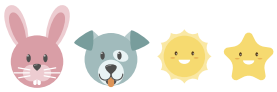
The correct answer is the second message.

The second message has exactly three animal stickers, so the rule about when to add a heart applies but the rule about when to add a star does not. Also, the animal stickers are not all different from each other, so the rule about when to add a sun does not apply.

All the other options are incorrect.

The first message has exactly two animal stickers, so the rule about when to add a star applies but the rule about when to add a heart does not. Also, the animal stickers are different from each other, so the rule about when to add a sun applies. If Amelia had sent the first message, it should look like this:

The third message has exactly two animal stickers, so the rule about when to add a star applies but the rule about when to add a heart does not. Also, the animal stickers are different from each other, so the rule about when to add a sun applies. If Amelia had sent the third message, it should look like this:



The fourth message has exactly three animal stickers, so the rule about when to add a heart applies but the rule about when to add a star does not. Also, the animal stickers are different from each other, so the rule about when to add a sun applies. If Amelia had sent the fourth message, it should look like this:

This is Computer Science!

This task's solution follows a simple algorithm to determine which messages were sent by Amelia, which consisted of successively testing each of Amelia's rules against the set of messages and collecting the set of messages that failed any of the rules. This implementation is an example of the use of if-then conditional statements in programming.

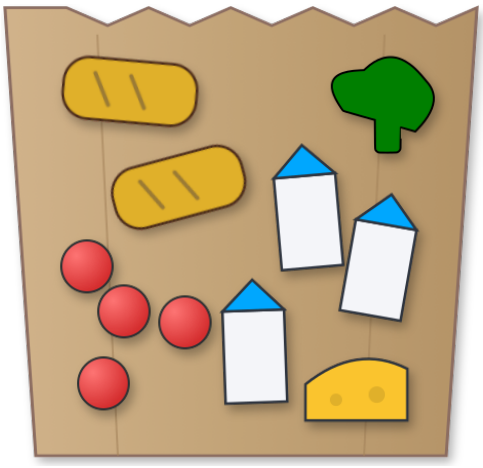
More generally, this task is an example of a data validation process, in which some inputs are analyzed to ensure that they conform to some pre-existing rules. In particular, the "stickers" attached to each input resemble the concept of a checksum used to verify the messages integrity.

Websites And Keywords

TODO

9 Shopping

Lalit went shopping.



BALAJI SUPERMART

ITEM CODE	PRICE
123484929354.....	₹10.00
123484920354.....	₹20.00
123484921354.....	₹30.00
123484922354.....	₹40.00
123484929354.....	₹10.00
123484921354.....	₹30.00
123484922354.....	₹40.00
123484929354.....	₹10.00
123484923354.....	₹50.00
123484929354.....	₹10.00
123484922354.....	₹40.00
TOTAL	**₹290.00**

ITEMS COUNT: 11

THANK YOU FOR SHOPPING!
Please come again.

What is the price of one bread  *?*

A) Rs.10

B) Rs.20

C) Rs.30

D) Rs.40

Solution

The correct answer is (C).

If we group the list of 11 prices in the receipt in terms of their frequency we get: 10, 10, 10, 10, 40, 40, 40, 30, 30, 20, 50.

The «Rs.10» must be the price of each red apple, because there are four apples and Rs.10 is the only price repeated 4 or more times. The «Rs. 40» must be the price of each milk carton, because after four Rs.10 are removed, Rs.40 is the only price repeated three or more times.

One might consider that the broccoli and cheese might have a price of Rs.30 each, but because there are two of item  , and Rs.30 is the only remaining repeated price, «Rs.30» must be the price of the  . Finally, the «Rs.20» and «Rs.50» must be the costs of the broccoli and the cheese (we cannot tell which is which from the information we have been given, but that's OK).

This is Computer Science!

This a task which we can apply the following informatics concepts:

This is a **constraint satisfaction** task where a valid solution must satisfy various conditions (constraints). In this task, we are presented with constraints in graphical form: various numbers of items and prices that must be matched with each other. This task is structured in a way (with different prices for different types of item) that there is a unique solution, and a greedy algorithm without backtracking will arrive at this solution. Specifically, there are two breads in the shopping bag, and there are exactly two copies of just one price «Rs.30».

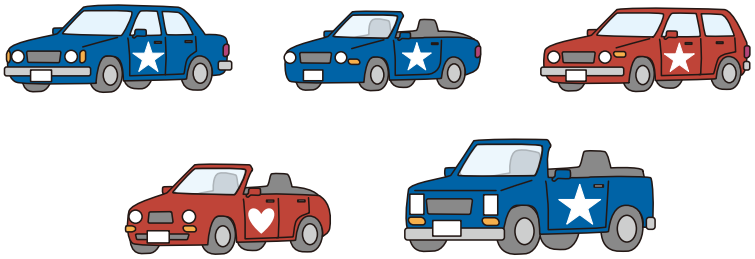
Sorting: Besides the proposed inference method, we can also efficiently match two lists. To do this, we can use sorting.

Websites And Keywords

TODO

10 Car Inspection Robot

A factory makes five different types of car and each one has a different shape.



Dr. Beaver wants a robot to identify each of these five car types. However, the robot is old and cannot recognize them by shape. It can only recognize three characteristics.

Which combination of three characteristics will allow the robot to tell all five cars apart?

A) Car color, door logo, and number of doors

B) Car color, door logo, and whether the roof is open

C) Door logo, number of doors, and whether the roof is open

D) Car color, whether the roof is open, and headlight shape

E) Door logo, number of doors, and headlight shape

Solution

Correct Answer: D) Car color, whether the roof is open, and headlight shape

A chart of the car features helps to check each choice.

	Car 1	Car 2	Car 3	Car 4	Car 5
The car color is blue	X	X			X
The logo is a star	X	X	X		X
The car has four doors	X		X	X	


Car 1


Car 2


Car 3


Car 4


Car 5

In the above table, Choice A cannot tell Car 2 and Car 5 apart.

Choice B cannot tell Car 2 and Car 5 apart.

	Car 1	Car 2	Car 3	Car 4	Car 5
The car color is blue	X	X			X
The logo is a star	X	X	X		X
The roof is open		X		X	X

Choice C cannot tell Car 2 and Car 5 apart.

	Car 1	Car 2	Car 3	Car 4	Car 5
The logo is a star	X	X	X		X
The car has four doors	X		X	X	
The roof is open		X		X	X

Choice D can identify all the cars.

	Car 1	Car 2	Car 3	Car 4	Car 5
The car color is blue	X	X			X
The roof is open		X		X	X
The headlights are round	X	X	X	X	

Choice E cannot tell Car 1 and Car 3 apart.

	Car 1	Car 2	Car 3	Car 4	Car 5
The logo is a star	X	X	X		X
The car has four doors	X		X	X	
The headlights are round	X	X	X	X	

This is Computer Science!

This task is about database design. To find exactly one piece of data from a collection, you need unique identification: a combination of features that is not repeated, called a candidate key.

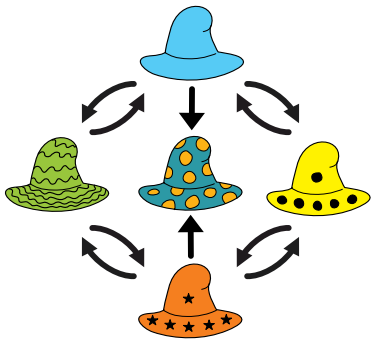
In this example, the combination of «car color,» «whether the roof is open,» and «headlight shape» serves as a candidate key to identify each of the five cars. Using these three features, any two cars can always be distinguished, allowing all five cars to be uniquely identified.

Websites And Keywords

TODO

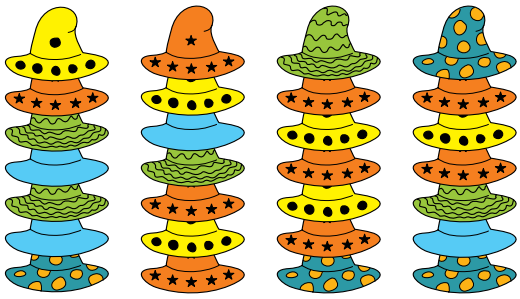
11 Tower of Hats

A hat shop owner likes to create attractive displays by arranging hats into towers. However, he does not want to stack the hats randomly. Instead, he asked an engineer to design an elegant arrangement scheme:



Each arrow shows which hat to pick up and place directly on top of the hat already there. For example, the scheme  shows that the yellow (left) hat is stacked on the right one: 

The shop owner intended to follow the scheme and built four towers of hats. However, he made a mistake in one of the towers. Which tower does not follow the scheme?



A) B) C) D)

Solution

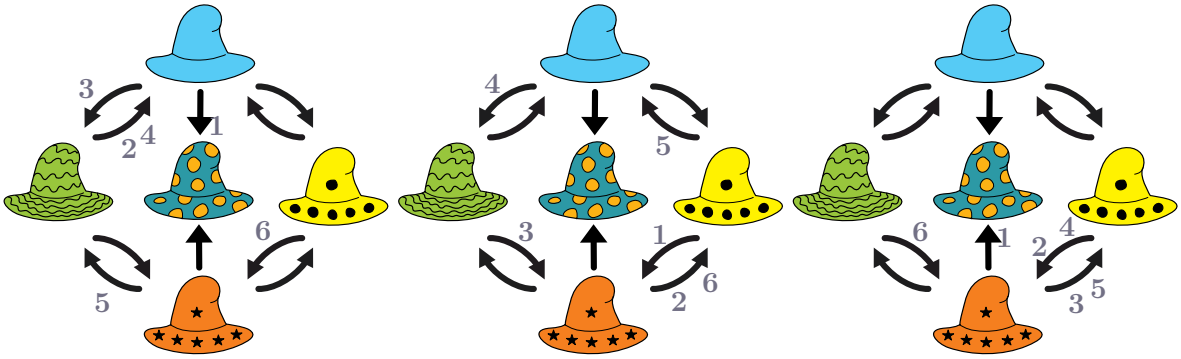
The correct answer is (D).

The scheme describes how hats must be arranged in a tower.

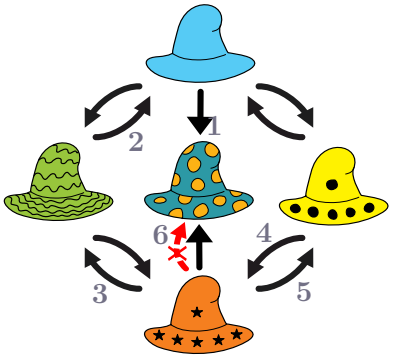
To solve the task, we need to carefully compare each tower with the scheme.

Starting from the bottom of the tower, we check whether the order and types of hats follow the rule shown in the scheme. Each level of the tower must match the corresponding rule.

Three towers follow the scheme correctly. In these towers, every hat is placed according to the required rule.



D. One tower breaks the rule at one of its levels. This tower therefore does not follow the scheme and is the correct answer.



Another way to analyze the solution is to realize that the bidirectional arrows allow either of the two hats next to them to be above or below. Therefore, you only need to consider the order of the hats where there are arrows in only one direction.

The only restriction, therefore, is that the blue hat with circles cannot be on top of the orange hat or the light blue hat. In fact, it cannot be on top of any of the other hats in the chart.

This is Computer Science!

In informatics, rules or instructions are often used to describe how objects should be arranged or processed.

Such rules can be written as algorithms or schemes that specify how elements should be organized. By following these rules step by step, a computer program can generate structures or verify whether a structure is correct.

In this task, the scheme represents a set of rules describing how hats may be stacked in a tower. The task is to check which tower follows these rules and which one does not. The scheme can also be interpreted as a simple automaton, a concept widely used in computer science. In an automaton, there are a number of states and the number transitions between states that are represented by arrows. Here, each type of hat corresponds to a state, and the arrows indicate which hat can be placed on top of another.

Some transitions can be repeated many times, forming loops. This means that certain hats can appear repeatedly in a tower as long as the stacking follows the allowed transitions.

Automaton and similar rule-based models are used in computer science to verify structures, recognize patterns, process sequences, and ensure that systems follow specified rules.

Websites And Keywords

TODO

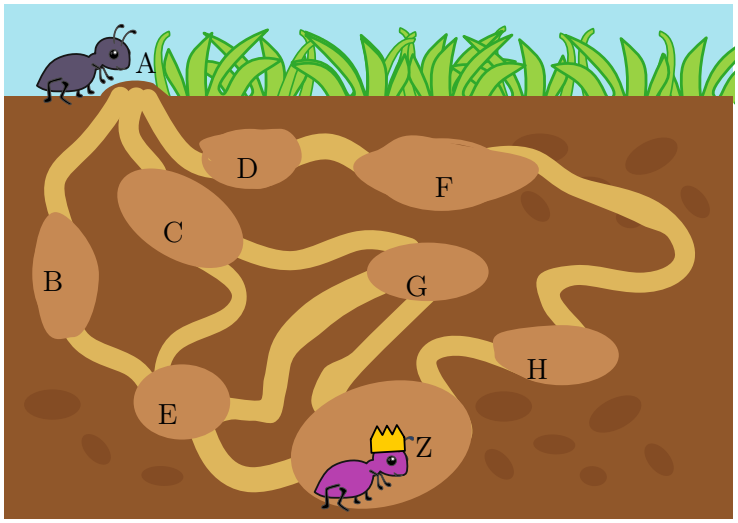
12 The Safe Tunnel

The ants use a complex network of underground tunnels to move around.

Recently, a very sneaky Spider has invaded the colony. She spins an **invisible web** to trap the passing ants. Nobody knows exactly where the Spider is hiding today, but the ants have noticed a pattern in her behavior:

The Spider is too afraid to go to the Entrance (A) or the Queen's room (Z), so those are always safe. However, she likes to place her webs **only** in busy rooms, the spots **where 3 or more tunnels meet**.

A Scout Ant needs to travel from the Entrance (A) to the Queen (Z) as shown in the map.



Based on the tunnel map, Select the path that guarantees the ant will arrive safely, no matter which of the busy rooms the Spider chose today.

A) A → C → G → Z

B) A → D → F → H → Z

C) A → B → E → Z

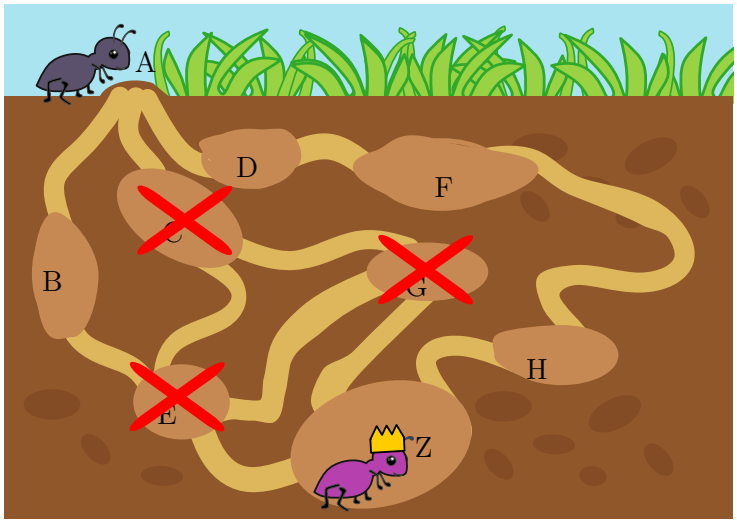
D) A → C → E → G → Z

Solution

The correct answer is (B).

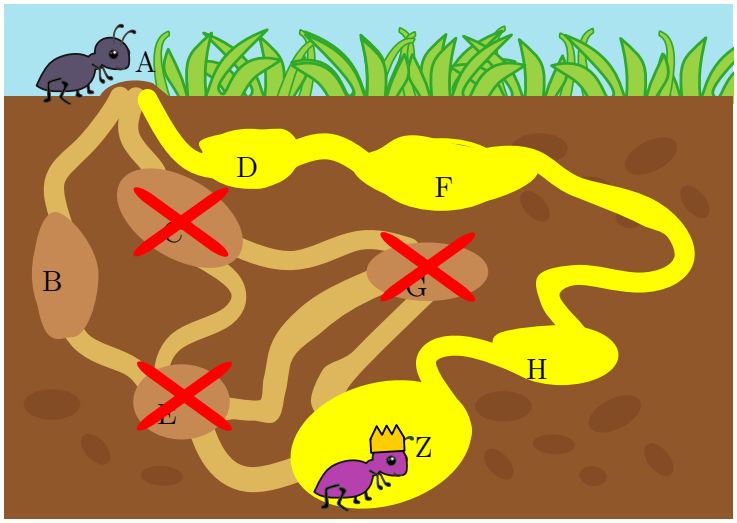
To find the safe path, we must count the number of tunnels connected to each room. The rules say the spider hides in busy rooms with 3 or more tunnels (Entrance A and Queen Z are safe). Let's look at the rooms we must avoid:

- Room C has 3 tunnels (it comes from A, and then splits to connect to both E and G
- Room E has 4 tunnels (connected to B, C, G, and Z).
- Room G has 3 tunnels (connected to C, E, and Z).



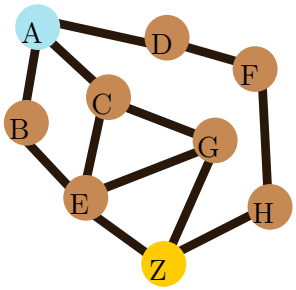
Any path going through these busy rooms is unsafe! Even though room B only has 2 tunnels and looks safe, it forces the ant to enter room E, which is a trap.

If we trace the outer path through rooms D, F, and H, we can count that each of these rooms has exactly 2 tunnels connected to them. Because they have 2 or fewer connections, they are completely safe from the spider, allowing the ant to reach the Queen.



This is Computer Science!

This task is about Network Security and Graph Theory.



Computer scientists use diagrams called Graphs to represent networks. Here is the graph of the ant tunnels. The circles (or rooms) are called **Nodes** and the lines are called **Edges** (like the tunnels or channels). The number of lines connected to a node is called the **Degree of the Node**.

Vulnerability Analysis: In real life, cybercriminals (hackers) often target central servers or «Hubs» (nodes with many connections) because that is where the most data traffic is concentrated. This is just like the Spider's invisible web in the busy spots.

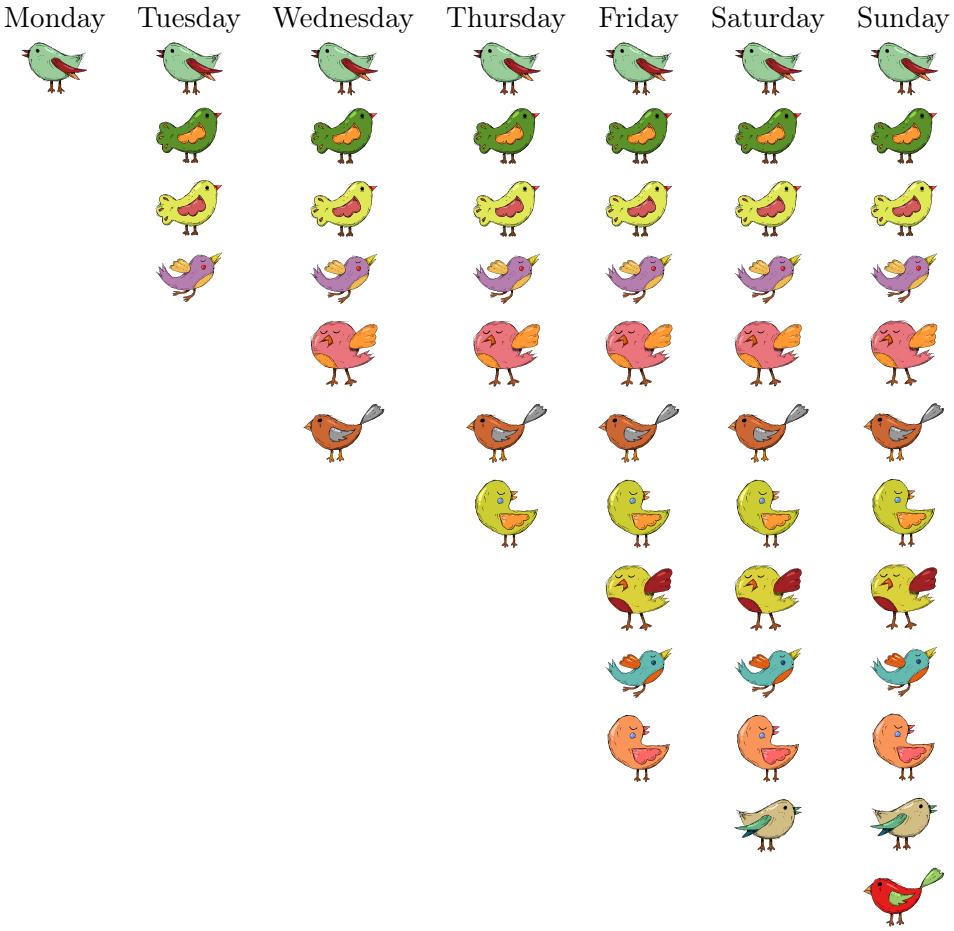
Secure Routing: When sending sensitive information, engineers sometimes design specific routes («Routing Algorithms») to avoid the most congested or public points of a network to reduce the risk of interception, even if the path is a bit longer. Like the ant had to do in this task.

Websites And Keywords

TODO

13 Bird Watching

A school environment club spent one week bird watching. They kept a list which they updated every time they saw a type of bird new to the club. This is what the list looked like after each day:



Some members of the club could only participate on four consecutive days.

- Anita was there from Tuesday to Friday.
- Badri was there from Wednesday to Saturday.
- Chetan was there from Monday to Thursday.
- Diya was there from Thursday to Sunday.

Which of these four members was present for the most sightings of a type of bird new to the club?

A) Anita
B) Badri
C) Chetan
D) Diya

Solution

The correct answer is (A).

To determine this, we can calculate how many sightings of a type of bird new to the club each person was present for. A nifty way to do this is to first calculate how many types of bird there are on the list at the end of each day:

Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
1	4	6	7	10	11	12

Now, remember that each person was present on consecutive days. This means we can calculate how many sightings of a type of bird new to the club a person was present for by looking at the total number of types of bird seen by the club before they participated and subtracting this from the total number of types of bird seen by the club after the last day they participated. Therefore,

- Anita first participated on Tuesday before which the club had seen 1 type of bird. She last participated on Friday after which the club had seen a total 10 types of bird new to the club. Therefore, Anita saw a total of 10−1=9 types of bird new to the club.
- Badri first participated on Wednesday before which the club had seen 4 types of bird new to the club. He last participated on Saturday after which the club had seen a total 11 types of bird new to the club. Therefore, Badri saw a total of 11−4=7 types of bird new to the club.
- Chetan first participated on Monday before which the club had seen 0 types of bird new to the club. He last participated on Thursday after which the club had seen a total 7 types of bird new to the club. Therefore, Chetan saw a total of 7−0=7 types of bird new to the club.
- Diya first participated on Thursday before which the club had seen 6 types of bird new to the club. She last participated on Sunday after which the club had seen a total 12 types of bird new to the club. Therefore, Diya saw a total of 12−6=6 types of bird new to the club.

We see that Anita was present for the most sightings of a type of bird new to the club.

This is Computer Science!

The number of types of bird new to the club seen by the club on each day form a sequence. That sequence is 1,3,2,1,3,1,1. (Can you see why?) Notice that the consecutive numbers 3,2,1,3 in this sequence correspond to Tuesday to Friday and 3+2+1+3=9 is the number of types of bird seen by Anita. However, it is interesting to note that we did not need to reproduce the sequence 1,3,2,1,3,1,1 in order to solve this problem. We calculated the result in a different way!

Instead, in the answer explanation, we just counted how many types of bird new to the clubs there were in the list at the end of each day. These counts are what we call partial sums. They are the running totals (sums) of numbers from the start of the sequence 1,3,2,1,3,1,1 up to each point in the sequence. Using subtraction, partial sums can be used to add up any consecutive numbers in the sequence – not only those from the start of the sequence.

In general, this technique of using partial sums can save a lot of time. This is especially true for very long sequences for which we want to calculate many sums of consecutive numbers in the sequence. The idea can leave to a clever way to develop an efficient algorithm.

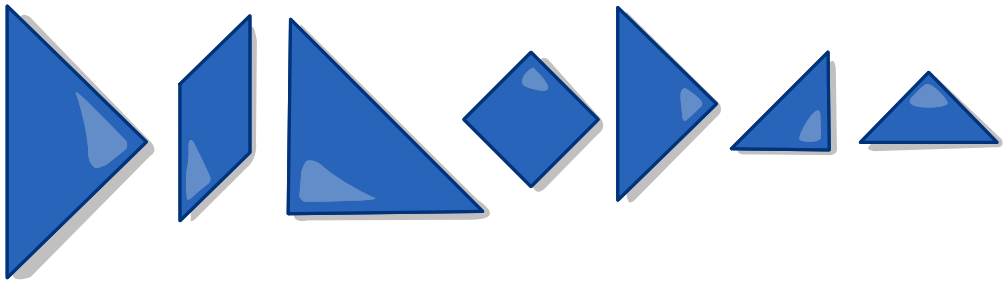
Separate from algorithms, the first step to solving this problem actually involves counting and storing or remembering how many types of bird are seen by the end of each day. As discussed, these counts form a sequence. A computer program would typically store the sequence using what is called an array. Because of how elements in an array are stored in a computer's memory, their values can be retrieved very quickly which is part of what makes the technique of partial sums particularly efficient.

Websites And Keywords

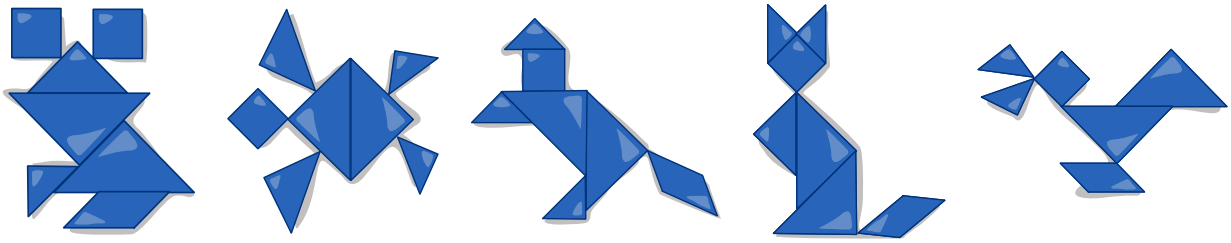
TODO

14 Beaver Puzzle

Two beavers, Geeta and Palak, are playing a puzzle together. The puzzle uses seven simple pieces. They have a picture sheet that shows different animals. One by one, they try to put together the animals using the seven pieces. The pieces can be rotated but cannot cover each other.



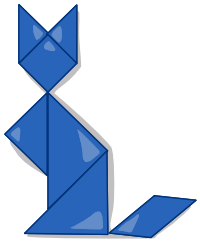
Which animal can they form using all seven pieces exactly once?



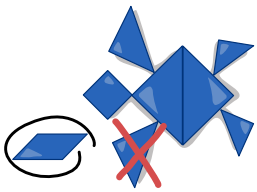
A) B) C) D) E)

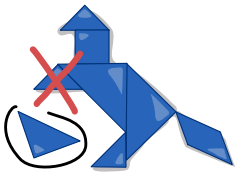
Solution

The correct answer is (D).


The cat needs 7 pieces: two large triangles, one medium triangle, two small triangles, one square, and one parallelogram.


Answer A is wrong: The frog needs 7 pieces: two large triangles, one medium triangle, one small triangle, two squares, and one parallelogram. So, they are missing one square and have one unused small triangle.


Answer B is wrong: The turtle needs 7 pieces: two large triangles, two medium triangles, two small triangles, one square. So, they are missing one medium triangle and have one unused parallelogram.


Answer C is wrong: The horse needs 7 pieces: two large triangles, three small triangles, one square, and one parallelogram. So, they are missing one small triangle and have one unused medium triangle.


Answer E is wrong: The bird only needs 6 pieces: two large triangles, two small triangles, one square, and one parallelogram. So, they have one unused medium triangle.

This is Computer Science!

The tangram task illustrates the principle of limited resources, which is also important in programming. There are exactly seven pieces available – no more and no less. Each of these seven pieces may be used only once. Any solution must work within these fixed constraints. In computer science, programs

must also operate with limited memory, limited processing time, or other restricted resources. The task demonstrates how a solution has to be found under clear resource constraints.

The tangram task can be viewed as a formal decision and search problem. It is a decision problem because we ask: Is there an animal that can be built using all seven pieces exactly once? It is also a search problem because we must find the exact arrangement of the pieces. The rules (each piece used once, no overlaps, exact fit) act as precise constraints. Modeling and systematically solving such well-defined problems is a central idea in computer science.

Websites And Keywords

TODO

15 Laptops

Four pupils attend an computer science course. At the end of class, they always store their laptops in two boxes. They stack the laptops carefully so that the next day, each pupil can grab their own directly off the top without having to move or rearrange any of the others.

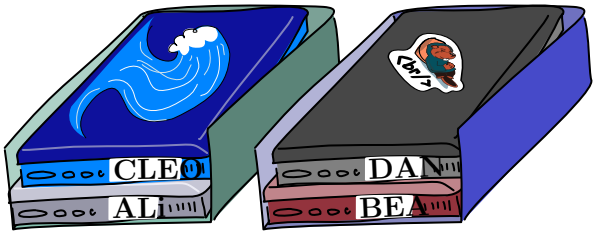
Today, they left the room in this order:

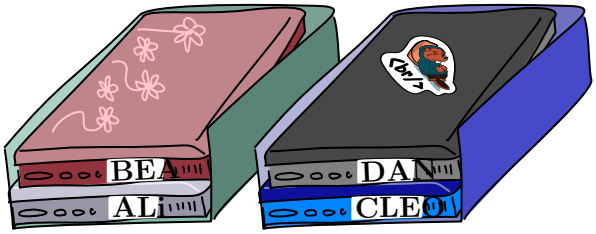
First Ali, then Bea, then Cleo, and finally Dan.

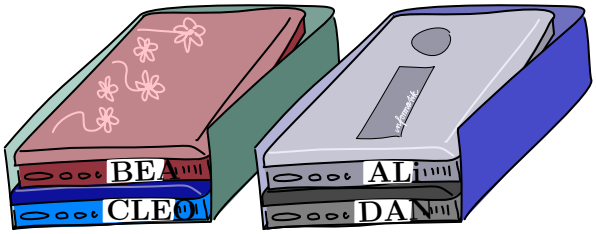
Based on their timetables, they already know they will arrive tomorrow in this order:

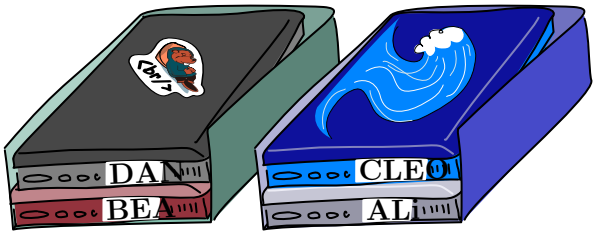
First Bea, then Ali, then Dan, and finally Cleo.

How did they arrange their laptops when they left the room?

A) 

B) 

C) 

D) 

Solution

B) is correct.

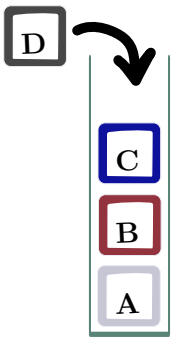
Bea is the first to enter the room. Since her laptop must be on the top of one of the two boxes, we can immediately discard (A) and (D). The next pupil to enter the room is Ali. He can pick his laptop from the bottom of the green box (C) or from the top of the blue box (B). In this step, we cannot rule out either of the two remaining options. Then, it is the turn of Dan. In either case, he can grab his laptop from the blue box: In (B), the laptop is the one on the top, while in (C) his laptop is the one on the bottom. Finally, Cleo can pick her laptop from the bottom of the green box (C) or from the bottom of the blue one (B). In these two last steps, we were not able to discard any of the two candidate solutions (B) and (C).

To resolve the issue, we need a tiebreaker between (B) and (C). To this end, we need to consider the order in which the pupils left the room. Ali was the first to leave. Therefore, his laptop must be on the bottom of either the green or of the blue box. This requirement is satisfied in option (B) but not in option (C). The correct answer is (B).

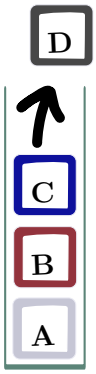
This is Computer Science!

In this Bebras task pupils put their laptops into two boxes. Laptops that were placed on top of a box last, can be grabbed first. In Computer Science, we call this kind of box a stack. Stacks are one of the most useful data structures when it comes to temporarily store data during a computation. Their underlying principle is called LIFO (last in, first out).

When using a stack, a program can perform two actions: it can push an element at the top of the stack, or it can pop the element that currently is on the top of the stack.



Dan is the last to put his laptop in the box.



Dan is the first who has to grab his laptop from the top of the stack. Bea has to wait until Cleo also grabbed her laptop before she can access hers.

This means that only the element at the top of the stack can be directly accessed at any given time. In order to access an element other than the one on the top, all the elements between it and the top of the stack must first be removed.

When working with exactly one stack, we can only pop elements in the reverse order of the push actions. With two stacks with two places each, four elements can be distributed among the stacks in six different ways (see picture at the end of answer explanation section).

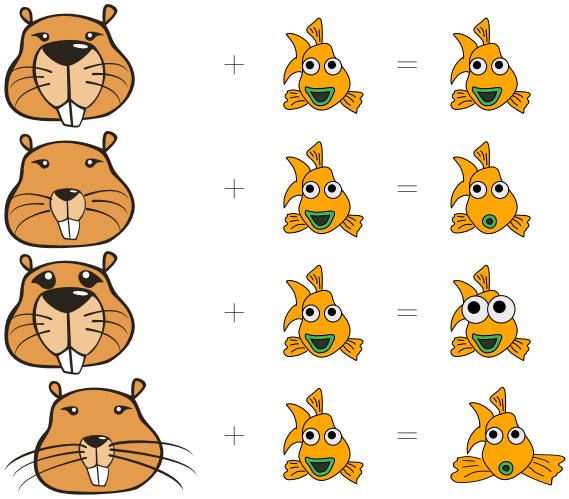
Websites And Keywords

TODO

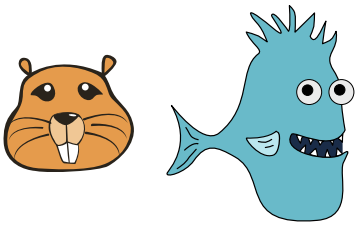
16 Beaverator

The Beaverator is a machine that uses two pictures as input: one of a beaver's face and one of a fish. The machine may change the fish picture. Then it outputs the resulting picture.

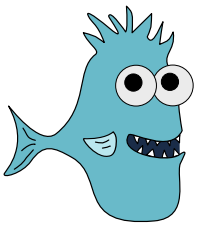
The Beaverator always follows the same simple rules to decide how to change the fish picture. The way a specific part of the beaver's face looks — for example its eyes — tells the Beaverator to change a specific part of the fish in a similar way. See four examples of the Beaverator's operation:



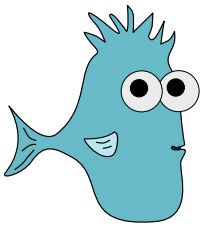
What picture will the Beaverator produce for these two inputs?



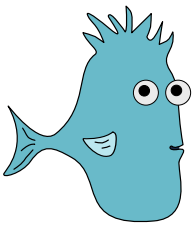
A)



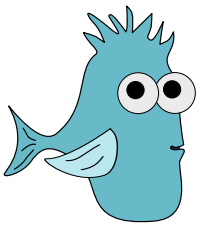
B)



C)



D)



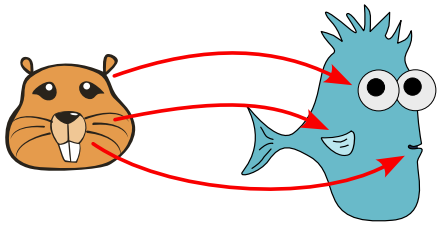
Solution

The Beaverator will output picture B.

As you can see from the examples in the task body, the Beaverator uses these rules:

- If the beaver has big eyes, the eyes of the fish get bigger.
- If the beaver has a small nose, the mouth of the fish gets smaller.
- If the beaver has long whiskers, the fins of the fish get bigger.

In the challenge, the beaver's face has big eyes and a small nose. After applying the above rules, the Beaverator outputs a picture of a fish that has bigger eyes and a smaller mouth than the fish in the input picture. Since the beaver's whiskers are normal-sized, the fish's fins remain unchanged.



The other options are wrong.

- A) The mouth of the fish is big.
- C) The eyes of the fish are small.
- D) The fins of the fish are large.

This is Computer Science!

In this task, the Beaverator combines two pictures to generate a new one. The fish picture provides the basic shape (content), while the beaver picture contributes visual features that modify the fish's appearance.

This idea is closely related to style transfer. Style transfer is a technique from generative AI in which a computer learns visual patterns—such as colors, textures, or brushstrokes—from many example images and then applies these patterns to new images. Similar to the Beaverator, a trained style transfer model receives two images as input: one that provides the style (for example, a painting such as the Mona Lisa by Leonardo da Vinci) and one that provides the content (for example, a photograph of a person). The model then generates a new image that shows the content image rendered in the artistic style of the painting.

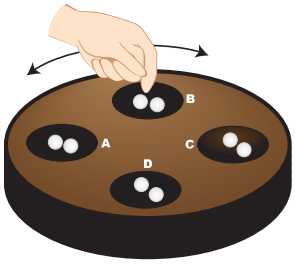
Unlike AI-based style transfer, however, the Beaverator does not learn from data. Instead, it operates according to simple, predefined rules.

Websites And Keywords

TODO

17 Pallanguzhi

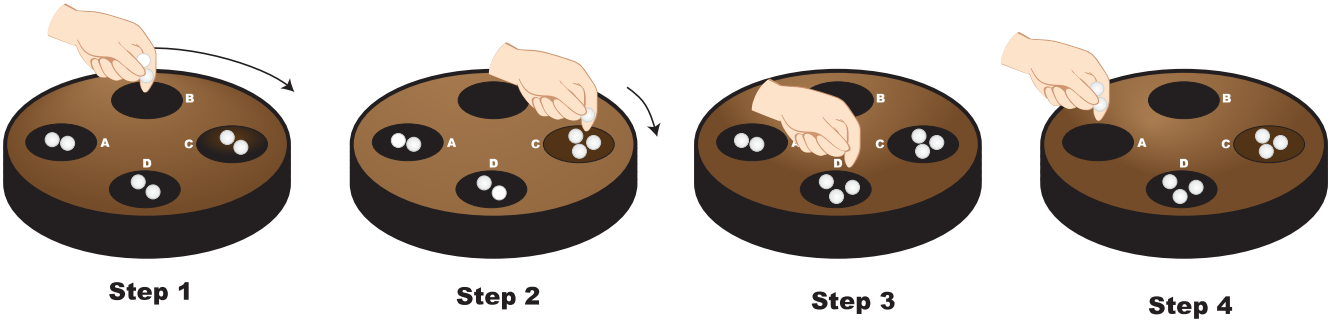
Chikki the beaver, loves playing «Pallanguzhi,» a traditional board game from India. The wooden board has 4 pits A, B, C, and D arranged in a circle. Each pit starts with 2 beads.



Chikki plays using these rules: 1: She chooses a pit. Then she picks up all the beads from it. 2: If the number of beads she picked up is even, she moves her hand around the board clockwise dropping one bead in each pit until her hand is empty. If it is odd, she moves her hand anti-clockwise dropping one bead in each pit until her hand is empty. 3: Then she looks at the next pit.

- If it has beads, she picks them up and continues from step 2.
- If it is empty the game ends.

For example:



If she picks up two beads from Pit B, she will move clockwise and drop one bead in Pit C, one bead in Pit D and stop. She will then look at pit A.

Chikki starts by picking beads up from Pit A. When the game ends, which pit has the most beads?

A) Pit A

B) Pit B

C) Pit C

D) Pit D

Solution

The correct answer is B) Pit B.

By following the rules step by step, the beads are redistributed over several rounds as shown in the table.

Action Phase	Pit A	Pit B	Pit C	Pit D	Stop Pit	
Initial Phase	2	2	2	2	-	Star
Round 1: Pick 2 beads from Pit A. Drop in Pits B, C.	0	3	3	2	Pit C	Pit D (ha
Round 2: Pick 2 beads from Pit D. Drop in Pits A, B.	1	4	3	0	Pit B	Pit C (has 3
Round 3: Pick 3 beads from Pit C. Drop in Pits B, A, D.	2	5	0	1	Pit D	Pit C

Since the pit next to where she stops (Pit C) is empty, the game ends.

Looking at the final counts, Pit B has the highest number of beads with a total of 5.

This is Computer Science!

This task shows how a process can repeat until a stopping condition is met. Each round follows the same rules, but the number of beads in the pits keeps changing.

In computer science, this kind of repeated action is common. A program often keeps running the same steps while a condition is true (In this case, while the next pit is not empty continue playing the game), and stops when the condition becomes false.

By keeping track of the changing number of beads in each pit across multiple rounds, we are «tracing» the state of a program, an essential skill computer scientists use to find bugs and understand how the program works.

Websites And Keywords

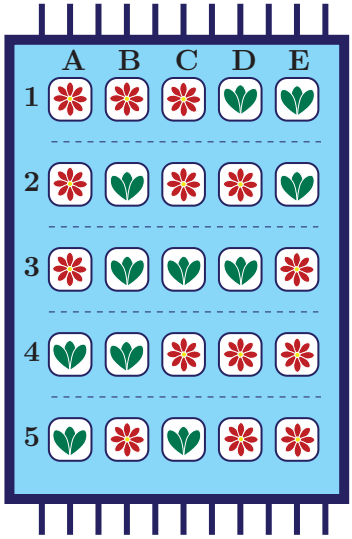
TODO

18 The Carpet Weaver's Mistake

Ali is learning to design Persian carpets. His teacher, Master Weaver, taught him two types of knots: "Flowers knot" and "Leaves knot" . Ali is trained by his Master by making a small 5-by−5 carpet. To obtain an aesthetic carpet, Master Weaver set two strict rules:

- Leaf Rule: Every Row must have an even number of Leaf knots (0, 2, or 4).
- Flower Rule: Every Column must have an odd number of Flower knots (1, 3, or 5).

Ali has finished his small carpet, but he accidentally used the wrong knot in exactly one cell.



Which knot did Ali get wrong?

A) Row 2, Column C

B) Row 3, Column B

C) Row 3, Column C

D) Row 4, Column B

Solution

The correct answer is B) Row 3, Column B.

Step 1: Check the Rows

According to the first rule, every row must contain an even number of Leaf knots. Therefore, each row should have 0, 2, or 4 Leaf knots. We check all rows to make sure that this condition is satisfied.

- Row 1: 2 Leaves (Even) — OK
- Row 2: 2 Leaves (Even) — OK
- Row 3: 3 Leaves (Odd) — ERROR!
- Row 4: 2 Leaves (Even) — OK
- Row 5: 2 Leaves (Even) — OK

=> Conclusion: The mistake is somewhere in Row 3.

Step 2: Check the Columns

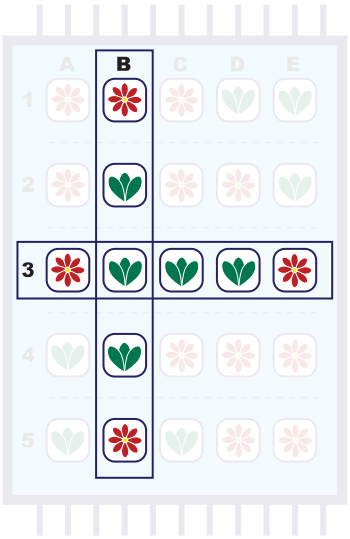
According to the second rule, every column must contain an odd number of Flower knots. Therefore, each column should have 1, 3, or 5 Flower knots. We check all columns to make sure that this condition is satisfied.

- Col A: 3 Flowers (Odd) — OK
- Col B: 2 Flower (Even) — ERROR!
- Col C: 3 Flowers (Odd) — OK
- Col D: 3 Flowers (Odd) — OK
- Col E: 3 Flowers (Odd) — OK

=> Conclusion: The mistake is somewhere in Column B.

Step 3: Locate the error

By combining these conclusions, the only cell that breaks both the first rule and the second rule is Row 3, Column B. If you swap that Leaf for a Flower, both rules will be satisfied.



This is Computer Science!

This task illustrates the concept of Error Detection using parity.

In computing, data is sent as a sequences of 1s and 0s. Sometimes, a bit gets flipped by accident (a «1» becomes a «0» or the other way round). To catch these mistakes, computers add a parity bit — an extra bit that makes the total number of 1s either always even or always odd. If one bit is flipped, this even-or-odd pattern is broken, so the computer can detect that an error has occurred.

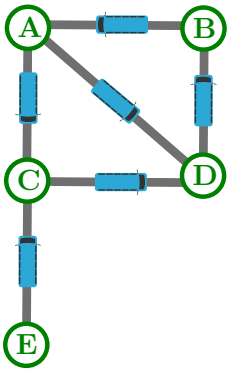
With such one-dimensional parity, the computer can detect an error, but it cannot identify exactly where the error is. By applying the same idea to both rows and columns, using a Two-Dimensional Parity Check, which is what is used in the task, makes it possible to identify the position of the flipped bit (if one bit has been flipped by error), making it possible to correct the error. In this task, an even parity is applied to rows and an odd parity is applied to columns. That is not required and the same parity could have been chosen for both rows and columns.

Websites And Keywords

TODO

19 Taxi

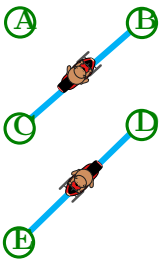
It's tourist season in Goa! There are five bus stops connecting the important locations, each marked with a letter. The map below shows the direct bus routes connecting these stops (in both directions).

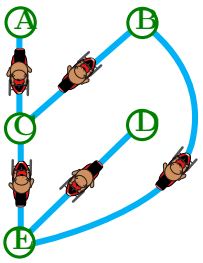


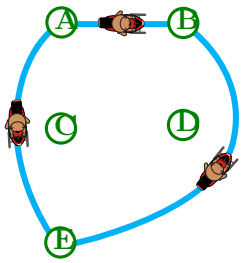
Borges's taxi service only travels between stops that are not directly connected by a bus route.

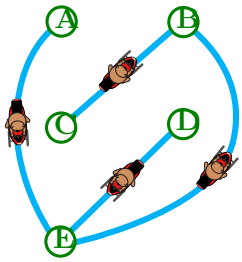
To help the tourists navigate Goa, he decides to draw a map of his taxi's routes.

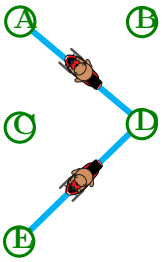
Which map correctly shows Borges's taxi routes?

A. 

B. 

C. 

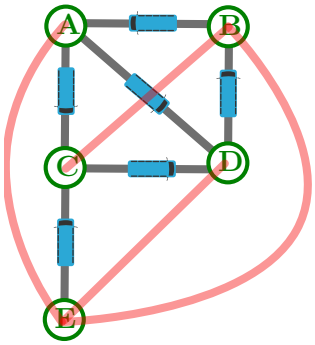
D. 

E. 

Solution

The correct answer is (D).

One approach to solve this would be by drawing lines for all missing direct connections between the stops on the bus map. To do this, pick one of the stops, then go through all other stops and see if there is a line; wherever there is no line, draw one. For example, let's pick stop B: there is a line going to A and D, but there are no lines going to C and E, so we draw those. Below is the resulting map.



We can also check that we aren't missing any lines by counting: There are 5 stops, so each stop needs to be connected to 4 other stops. For example, stop A has 3 gray lines and 1 red line, so 4 lines in total.

Once we have drawn all missing connections, the red lines (and the bus stops) correspond to Borges's taxi map, which is the option D.

This is Computer Science!

The map of bus stops in this task is called a graph in informatics.

Graphs are a powerful tool for representing networks (like our map, or computer networks), relationships (like friendships on social media), or links between websites. In informatics, analysing what is not connected can be just as important as analysing what is. Graphs consist of:

- Nodes: The individual items or locations (in our case, the bus stops).
- Edges: The lines connecting the items (in our case, the bus routes). This is where complement graphs come in. A complement graph takes the exact same nodes as the original graph but flips the connections:
- It only draws edges where the original graph did not have them.

Borges's map is a complement graph to the regular bus map!

Interestingly, some problems that are challenging to solve on the original graph can become easier when approached using its complement. Some algorithms may run more efficiently on the complement graph.

In parallel computing, tasks often have dependencies that prevent them from running simultaneously. These dependencies can be modelled as a graph:

- Each node represents a task.
- An edge between two nodes indicates that the tasks cannot run at the same time (because one depends on the other).

The complement graph, in this case, shows which tasks can run simultaneously, that is, tasks that are not connected by a dependency. By identifying these independent sets of tasks, the computer can execute them in parallel, ultimately speeding up computations.

Websites And Keywords

–

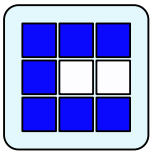
20 Magic Spy Sheets

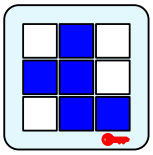
Agent Fox needs to deliver a Secret Map to his base. To hide the information from spies, he uses a system with two clear plastic sheets:

- A **Key Sheet**, which is kept safely at the base and
- A **Message Sheet**, which is the one he will actually send.

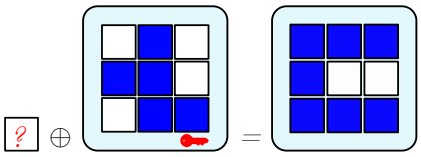
When the base receives the Message Sheet, they stack it on top of the Key Sheet. Then the squares appear magically like this:

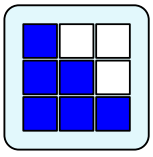
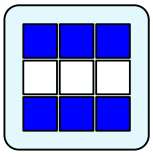
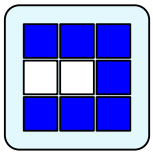
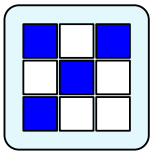
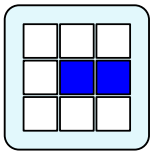
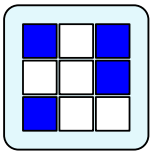
- Blue ■ ⊕ Clear □ = Blue ■
- Clear □ ⊕ Blue ■ = Blue ■
- Clear □ ⊕ Clear □ = Clear □
- Blue ■ ⊕ Blue ■ = Clear □ (Magic! The color disappears!)

Agent Fox needs to deliver this Secret Map: 

And the base will use this Key Sheet: 

Which Message Sheet should Agent Fox send?



A.  B.  C.  D.  E.  F. 

Solution

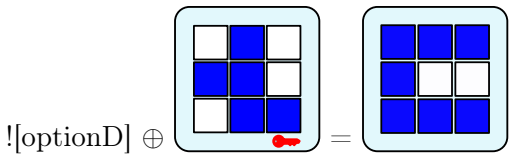
Option D is the correct answer.

To find the Message Sheet, we need to figure out which squares will combine with the Key Sheet to create the Map.

For each position, we know the Key Sheet and the Secret Map colors. By matching them to our 4 rules, we can easily figure out the color of the missing Message Sheet square:

Square (Position): Message Sheet (To Find ?) + Key Sheet (Given) = Secret Map (Goal- Given)
A (Top-left): Blue ■ ⊕ Clear □ = Blue B (Bottom-right): Clear □ ⊕ Blue ■ = Blue ■
C (Middle-right): Clear □ ⊕ Clear □ = Clear □ D (Center): Blue ■ ⊕ Blue ■ = Clear □ (Disappears!)

When applying this logic to all squares, we find that:



You can also simplify the rules into an easy trick: If the two squares are different (Blue ■ ⊕ Clear □ or Clear □ ⊕ Blue ■), the result is Blue ■.

If they are the same (both Blue ■ ⊕ ■ or both Clear □ ⊕ □), the result is Clear □ .

This is Computer Science!

This magic rule is actually a common computer operation called XOR (Exclusive OR).

The interesting thing about XOR is that it is completely reversible.

If for example Secret Message □ ⊕ ■ Key = ■ Secret Map, then applying the same rule backwards gives you the answer: ■ Secret Map ⊕ ■ Key = □ Secret Message .

Computers often protect information using a method called encryption, which works in a similar way. Instead of blue and clear squares, computers use 1s and 0s. The message is a sequence of bits, and the key is another sequence of bits.

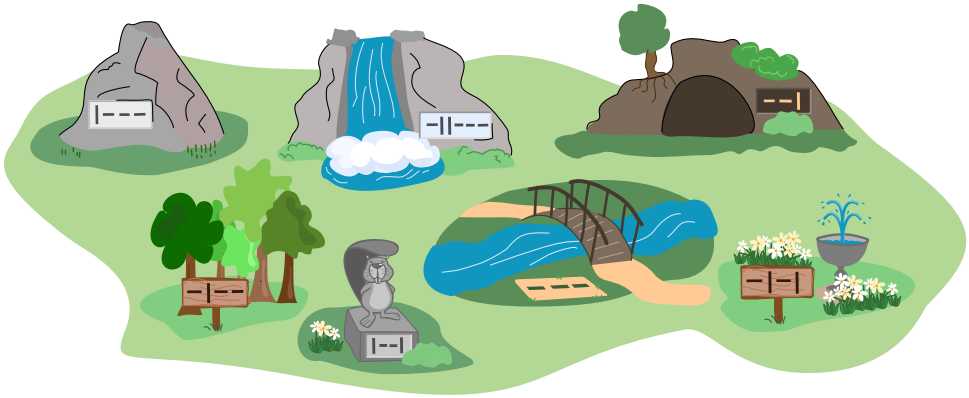
- $1 \oplus 0 = 1$ (Visible) / $0 \oplus 1 = 1$ (Visible)
- $1 \oplus 1 = 0$ (Invisible!) / $0 \oplus 0 = 0$ (Invisible!)

Using the XOR operation, messages can be encrypted so that if someone steals the Secret Message, they only see random 0s and 1s. To reveal the real message, the encryption key is needed; without it, the operation cannot be reversed. This is because the result is ambiguous: seeing a 0 or a 1 does not tell you which values were used before unless you also know the key. Visual cryptography uses the same idea: one image can hide another inside it, and the hidden image becomes visible only when the correct decryption pattern is applied.

Websites And Keywords

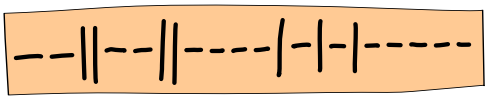
TODO

21 Ben's Walk



Ben took a walk through the park and visited several sites.

Each site has an ID made from the two symbols – and |. As he walked, Ben recorded the IDs on the sheet below, from left to right, in the order he passed the sites.



Ben did not pass any site more than once.

Which sequence of sites did Ben visit?

A) Cave → Rock → Beaver Statue → Fountain → Trees → Bridge

B) Cave → Beaver Statue → Rock → Fountain → Trees → Bridge

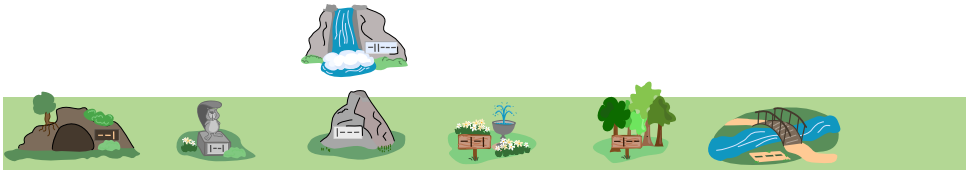
C) Cave → Beaver Statue → Fountain → Trees → Rock → Bridge

D) Cave → Beaver Statue → Rock → Trees → Fountain → Bridge

Solution

Correct Answer: B

The route that he passed would be:



Each site is identified by an ID with different symbols. Let us check Ben's record of his walk.

– – | | – – | | – – – – | – | – | – – – – –

The first three symbols – – | only match the ID of the cave. None of the other IDs start with this sequence. Therefore, the cave must be the first site Ben passed by.

The next symbol in the record starts with |. Only IDs | – – | and | – – – start with I, and only | – – | matches the sequence in Ben's record. So the beaver statue came next on Ben's walk.

The third site on Ben's walk is the rock | – – –, as it is the only ID left that starts with a |

The fourth is the fountain – | – |. This is because all ID´s are at least three characters. Both the fountain and the trees starts with – | –. However, looking at the fourth symbol in the ID, we can determine that it must be the fountain as it ends with |.

The next stop is the group of trees – | – –. This share the same start as the previous stop (– | –). Since we already have the fountain, only the trees are left with this start.

The walk finally ended at the bridge – – –, as there are only three characters left and they fit the bridge´s ID. T he recorded sheet contains the waterfall ID – | | – –, but the preceding part of the string – – | | – does not match any sequence of other site IDs. That is why we can tell that Ben did not pass by the waterfall.

This is Computer Science!

You have just decoded a sequence of code words from a continuous code string without separators between the code words. The decoding was possible mainly because the code (the set of site IDs) in this Bebras task is prefix-free: No codeword is a prefix of any other (meaning that no codeword starts with any other codeword).

Prefix-free codes play a crucial role in informatics, mainly for data compression. Their defining property guarantees unique decodability: When a stream of code symbols – in computing, code symbols are bits very often – is read sequentially from left to right, the boundaries between codewords are unambiguous. At every position in the stream, it is clear where one code word ends and the next begins, without the need for separators or lookahead.

A well-known example for generating prefix-free codes is Huffman coding, developed by David A. Huffman in 1952. This widely used compression method assigns shorter codewords to more frequent symbols and longer codewords to less frequent ones. Huffman codes are specifically constructed to be prefix-free, ensuring that compressed data can be decoded reliably and efficiently.

Websites And Keywords

TODO

22 Video Recommendation

A video app uses a smart rule to suggest new videos:

If another person watched at least 2 of the same videos as you, the app suggests their other videos to you (if you have not seen them).

You have recently watched these three videos:


How to cook eggs


Italian Pasta Recipe


Burger Recipe

Here is what three other people watched:

Anu


Italian Pasta Recipe


Burger Recipe


Salad Recipe

Bela


How to cook eggs


Baking Bread


Burger Recipe

Charu

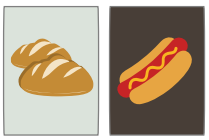

Italian Pasta Recipe


Baking Bread

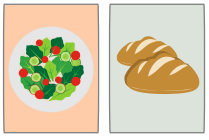

Street Food

According to the rule, which new videos will the app recommend to you?

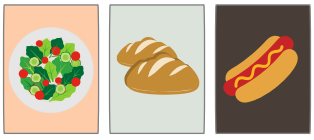
A)



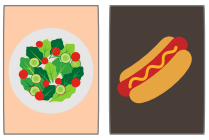
B)



C)



D)



Solution

Correct answer is B.

The suggested videos would be  and  .

Anu watched 2 videos that you also watched  and  . She meets the rule! So, the app recommends her other video: 

Bela also watched 2 videos you watched  and  . She meets the rule too! The app recommends:  .

Charu only watched 1 video you watched  . She doesn't meet the "at least 2" rule, so you don't get her videos.

This is Computer Science!

This task shows how apps like Netflix, YouTube, or Spotify know what to suggest to you. This is called collaborative filtering.

The computer looks for "similar persons" who like the same things as you. If they watched something you haven't seen yet, the app suggests it!

While this is great to find things you like, it can also trap you in a filter bubble. This means the app only shows you things you already know, and you might miss out on completely new and different ideas.

Websites And Keywords

TODO

23 Beaver Dam Repair

After a storm, part of the beaver dam has collapsed. Five beavers Asha, Bimal, Chitra, Dev, and Esha rush to carry out repairs.

Each beaver performs exactly one task, and each task is performed by exactly one beaver. The repair tasks are:

- Stabilizing logs
- Redirecting mud
- Cutting wood
- Fixing joints
- Clearing debris

Each beaver knows how to do two of the tasks, as shown in the table below.

Beaver	**Tasks they can perform**
Asha	Stabilizing logs, Redirecting mud
Bimal	Stabilizing logs, Cutting wood
Chitra	Clearing debris, Redirecting mud
Dev	Fixing joints, Cutting wood
Esha	Clearing debris, Fixing joints

Which of the following assignments is not possible?

A) Chitra clears debris and Dev cuts wood

B) Esha clears debris and Bimal cuts wood

C) Asha stabilizes logs and Dev fixes joints

D) Esha clears debris and Bimal stabilizes logs

Solution

The correct answer is D.

We can examine how the assignments are determined using the example below.

Asha can either stabilize logs or redirect mud. First, suppose that Asha stabilizes logs. Then the only beaver left to redirect mud is Chitra; the only beaver left to clear debris is Esha; the only beaver left to fix joints is Dev; and the only beaver left to cut wood is Bimal.

Second, suppose that Asha redirects mud instead. Then the only beaver left to stabilize logs is Bimal; the only beaver left to cut wood is Dev; the only beaver left to fix joints is Esha; and the only beaver left to clear debris is Chitra.

There are no other possible assignments of jobs to beavers.

Thus, we can see that:

- answer option A is consistent with the second assignment;
- answer options B and C are consistent with the first assignment;
- answer option D is consistent with neither.

This is Computer Science!

This task represents a constraint satisfaction problem (CSP), in computer science.

- There is a fixed set of variables (the beavers).
- Each variable must be assigned one value from a limited domain (the tasks the beaver can safely do).
- The assignments must satisfy global constraints:
  - each task is assigned to exactly one beaver,
  - each beaver gets exactly one task,
  - only allowed task-beaver combinations are permitted.

Such problems arise in computer science in areas like:

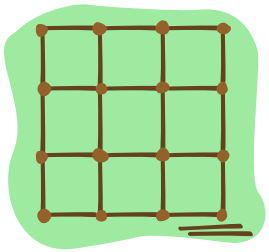
- job assignment in operating systems,
- resource allocation in distributed systems

Websites And Keywords

TODO

24 Muddy or Clear?

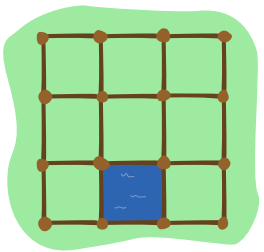
The beavers play a game with clear and muddy water. First they build a grid like this using posts and removable walls:

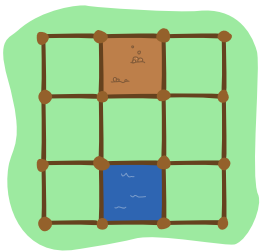


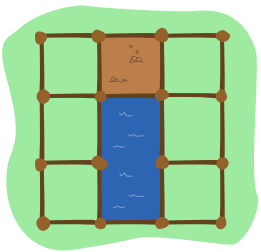
After, they start playing, they always follow these rules:

- Luca pours clear water into one randomly chosen field.
- Mora pours muddy water in any other field.
- Luca and Mora then take turns removing one wall at a time. Luca starts first. When a wall is removed, the player's water flows into a neighboring field.
- A wall may not be removed if:
  - it is an outer wall of the playing area;
  - there is no water on both sides;
  - it has muddy water on one side and clear water on the other side.
  - The player who cannot remove the wall loses the game.

Here an example of first 3 turns:


Luca

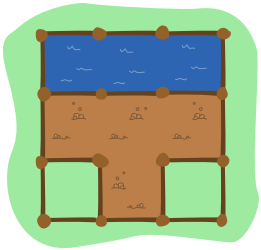

Mora

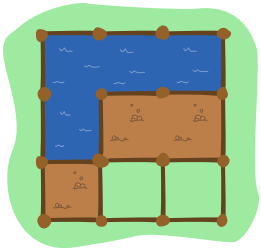

Luca

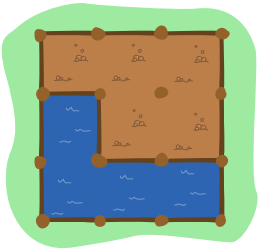
...

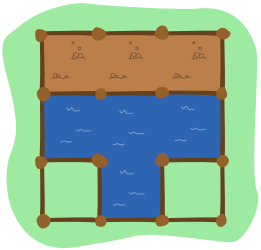
Mora

Only one of these states is possible in the game. Which one?

A) 

B) 

C) 

D) 

Solution

D is the correct option.

Since Luca goes first, there can not be more fields flooded with muddy water than clear water.

Answer A) is wrong because they have more fields with muddy water than clear water.

Answer B) is wrong because a wall has to be removed so that the water flows into the neighbouring field. Fields that are not directly adjacent cannot be flooded.

Answer C) is wrong because they have more fields with muddy water than clear water.

Answer D) is correct. Luca has made her move and Mora can make no more moves and the game ends.

This is Computer Science!

In this task, you need to find conditions that never change during the game. Since these conditions are true before and after each move, they will also be true when the game ends! Before Luca moves there is an equal amount of clear water and muddy water on the board. After Luca moves there is one extra cell of clear water compared to the muddy water. All the clear water cells must touch each other in one big group, and all the muddy water cells must do the same. The conditions that never change during the game are called invariants.

In programming, an invariant is a condition or constraint that remains consistently true throughout the execution of a specific section of code. For example, a condition that is true before and after every iteration of a loop is called a loop invariant. It's the same thing for this game.

If we follow certain rules, the resulting outcome will make those conditions visible. Conversely, to achieve a specific outcome, we must clearly define the rules and the sequence of actions (algorithm). By comparing our goal with what actually happened, we can find the errors in our logic.

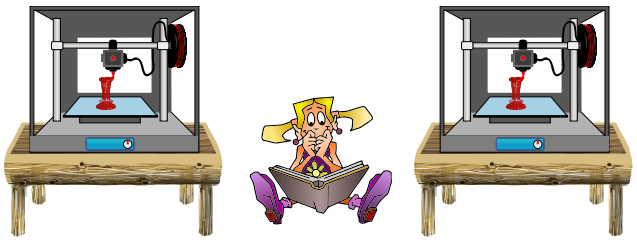
In this task, you excluded the impossible outcomes and found out which invariant was not followed in each case. You can also use this approach when looking for an error in your algorithm (or your program). Computer scientists call this debugging. One debugging strategy is to analyze the results (or meaningful intermediate results) to determine where the error occurred.

Websites And Keywords

–

25 3D Printing

A town library has two 3D printers that are available for people to use when they are in the library. If a person arrives and a printer is available, he or she starts their print job immediately. Otherwise that person must wait until a printer becomes available. If more than one person is waiting, when a printer becomes available, the next person is chosen randomly by the librarian.



One day 6 people wanted to use the 3D printers. Their arrival time and the time it will take to complete their print job is shown in the table.

Person	Arrival Time	Length of Print Job
Abby	8:00 am	5 hours
Bojan	10:00 am	3 hours
Chloe	12:00 pm	2 hours
Dipika	1:00 pm	4 hours
Edi	2:00 pm	1 hour
Foster	2:00 pm	6 hours

What is the earliest possible time that the last print job will finish?

A) 8:00 pm

B) 9:00 pm

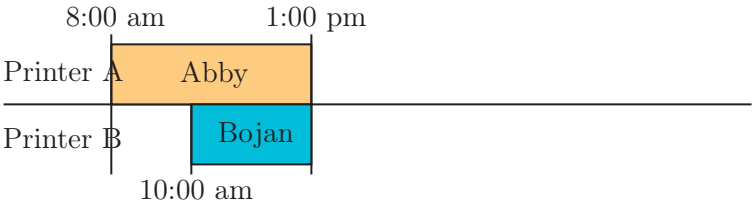
C) 10:00 pm

D) 11:00 pm

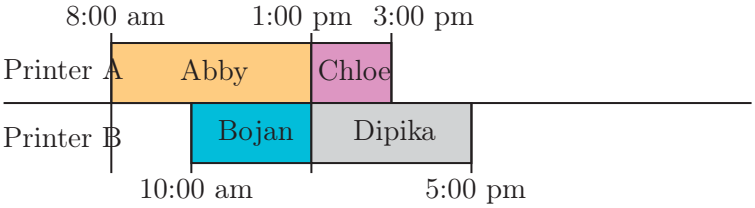
Solution

The correct answer is (B) 9:00 pm.

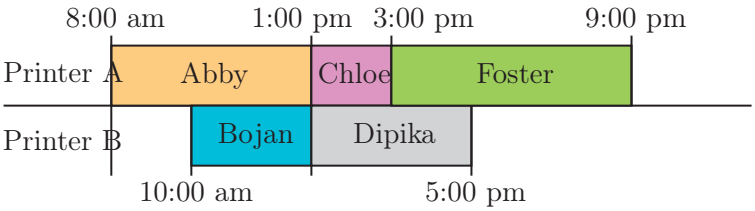
First we will call the printers Printer A and Printer B. When Abby arrives at 8:00 am the printers are both free so she begins printing immediately. Suppose she uses Printer A. Her print job will finish at 1:00 pm. When Bojan arrives at 10:00 am Printer B is free so he begins printing immediately. His print job will also finish at 1:00 pm. These print jobs are shown in the following diagram.



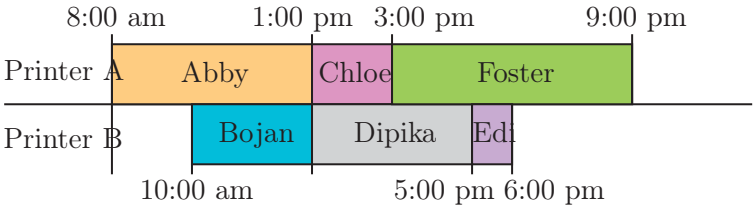
When Chloe arrives at 12:00 pm both printers are in use so she must wait for one to become available. When Dipika arrives at 1:00 pm both printers have just become available. Since there are only two people waiting, they can both start their print jobs immediately. Suppose Chloe starts her print job on Printer A and Dipika starts her print job on Printer B. Chloe's print job will finish at 3:00 pm and Dipika's print job will finish at 5:00 pm. The updated diagram is shown.



When Edi and Foster arrive at 2:00 pm, both printers are in use so they must wait for one to become available. At 3:00 pm, Chloe's print job finishes so Printer A is available. Either Edi or Foster could start their print job, but since we want to know the earliest possible time that the last print job will finish, and these are the last two people, the person with the longer print job should go first. Thus, Foster starts his print job on Printer A. His print job will finish at 9:00 pm. The updated diagram is shown.



At 5:00 pm, Dipika's print job finishes so Printer B is available. Edi starts his print job and it will finish at 6:00 pm. The updated diagram is shown.



Thus, Foster's print job is the last to finish at 9:00 pm.

Note that 3:00 pm was the only time when we had to make a choice about who should start their print job next. If instead we had chosen Edi to start his print job on Printer A, then his print job would finish at 4:00 pm. Then since Printer B is still in use, Foster would start his print job on Printer A, and it would finish at 10:00 pm. This is later than the finish time we already found. Therefore, the earliest possible time that the last print job will finish is at 9:00 pm.

This is Computer Science!

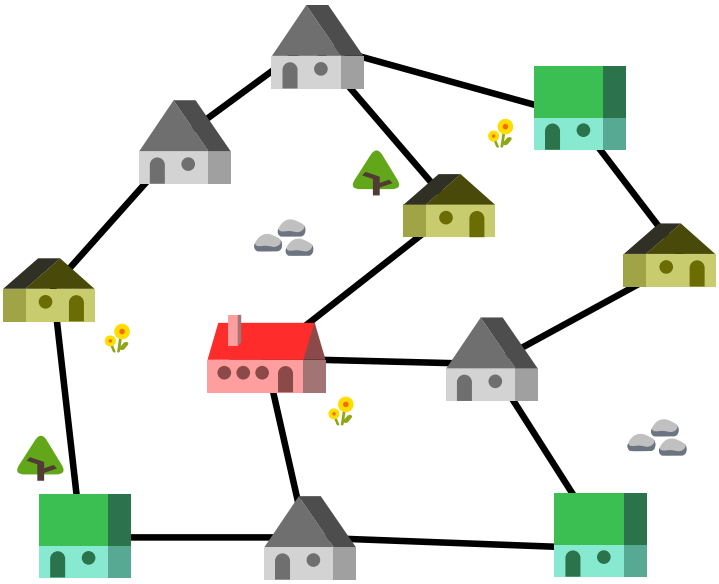
This task is an example of parallel computing. Parallel computing is a method that breaks tasks into smaller parts and then completes the smaller parts simultaneously on multiple processors. In this situation, the task is all the print jobs for the day, and the two 3D printers act as processors because they can execute different print jobs at the same time. Parallel computing increases efficiency since multiple operations run at the same time.

This problem asks to minimize the makespan, which is the time elapsed from the start of the first task to the completion of the last task in a project. We can minimize the makespan through the use of good scheduling, which is deciding how and when tasks are assigned. If scheduling is done poorly, jobs can take longer to complete tasks and resources can end up being wasted. Thus, good scheduling is important to maximize efficiency.

Websites And Keywords

TODO

26 Gifts for Grandchildren



The picture above shows a map of the houses and roads in the village. Each road (black line) between two houses is exactly 1 km long.

Grandma lives in the big (red) house  , and her grandchildren in the smaller (yellow) houses  . The green houses  are toy stores and the grey houses ![greyh] are where other people live.

Grandma wants to buy gifts at (exactly) one of the toy stores and deliver them to her grandchildren in all three yellow houses.

What is the minimum distance Grandma needs to travel (in km) that takes her from her house to one of the shops, then to each of the three houses of her grandchildren (to deliver the toys) and then back again to her house?

A) 9 km

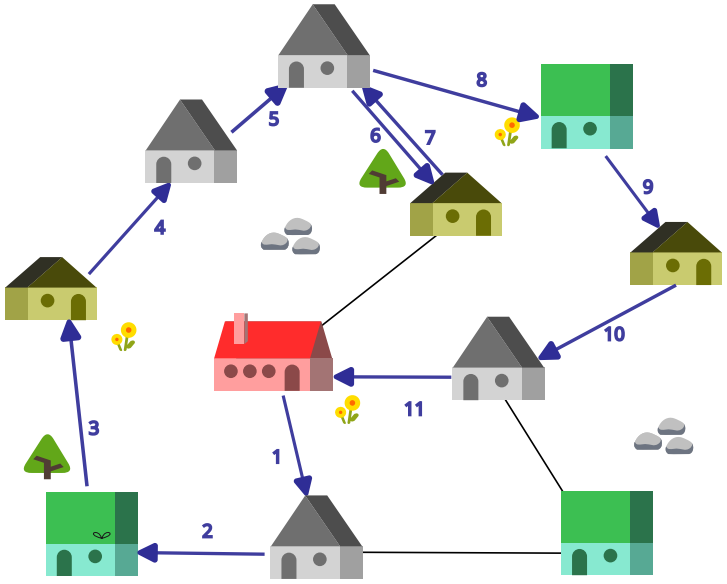
B) 10 km

C) 11 km

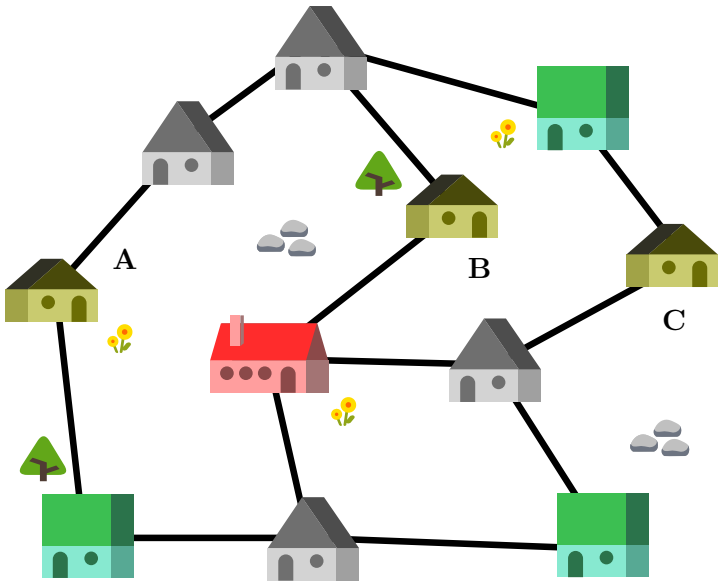
D) 12 km

Solution

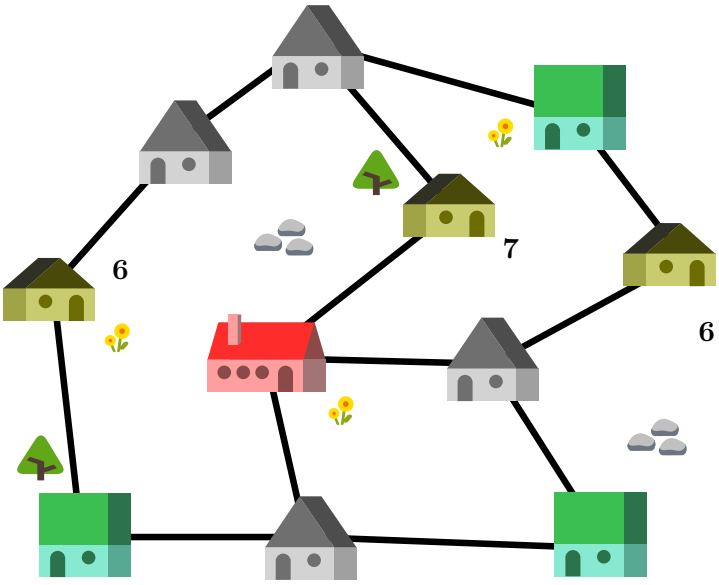
The correct answer is 11 km. One example of how she can travel is shown in the picture below (but there are other solutions that accomplish the same distance).



To see that you cannot get the task done by travelling less than 11 km, we can proceed as follows;

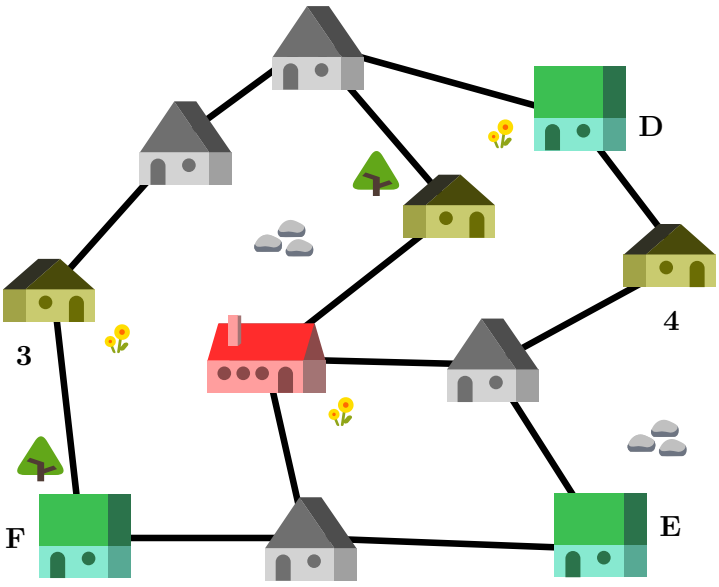


- Let's first concentrate on visiting the 3 houses of the grandchildren (A, B and C in the picture on the left). The distance between B and C is 3 km (there are two paths you can use), between A and B is 3 km, and between A and C 4 km.



It is fairly easy to see that you can visit all three houses using only 6 km, starting at A or C and then you will end up at C or A. If you want to end up at B, you need 7 km. We indicated this in the picture below

- After visiting the three houses, you need to return home. If you ended up at B, you need only 1 more kilometer. If you ended up at A or C, you need at least 2 more km. In any case, that means that you need at least 8 km to visit all grandchildren and return back to Grandma's house.



- Now let's tackle the problem from the other side. How many kilometers do you need to go from Grandma's house to one of the shops and then to one of the grandchildren's houses?

This depends on the shop chosen. If you choose shop F (see picture on the right), you need 3 km to end up at A. If you choose shops D or E, you need 4 km to end up at C.

So you need at least 3 km for the first part of the journey.

Adding the 8 km you need for the second part, you need a total journey of at least 11 km!

This is Computer Science!

This task is a good example of how to translate an everyday situation into the language of computer science and solve it using well-known models in computer science. From the point of view of computer science, it is not just about the shortest path between two points, but about planning a route with multiple objectives and constraints that must be satisfied in the correct order. Such problems belong to optimization on graphs with constraints.

Graphs are computer scientist's way to represent connections between objects – here roads between houses. They are used to model many real life problems, like finding the shortest path between two places on a map.

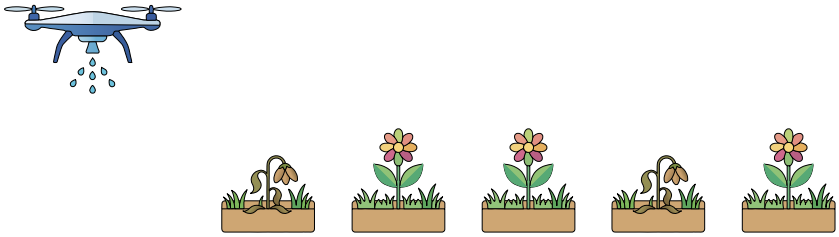
This task is a little more involved: you cannot simply look for the shortest path that contains Grandma's house, one shop and the three houses of the grandchildren, because the order in which you visit the places on the map, is very important here: you have to visit a shop first. So, some times computer scientists need to adapt well-known algorithms (find the shortest route) to the specific problem at hand.

Websites And Keywords

TODO

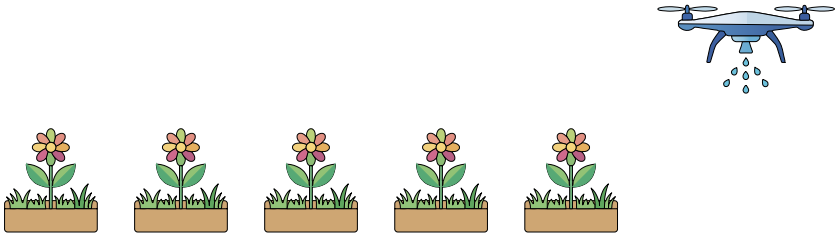
27 Watering Drone

Dr. Beaver tried to develop a drone that automatically waters wilted flowers to make them bloom. She is not entirely sure that it works perfectly.



Dr. Beaver programmed the drone as follows. When the drone starts above the leftmost flower box, it repeats the following instructions in order until it stops:

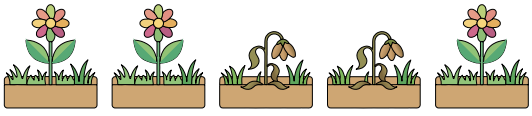
- Check the box directly below. If there is a wilted flower in it, water it. Otherwise, move to the box on the right.
- If the drone is now no longer over a flower box, stop.
- Move to the flower box on the right.

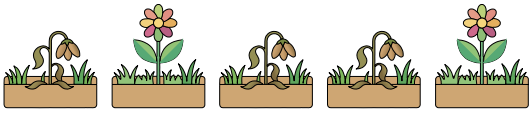


Dr. Beaver suspects that the drone sometimes fails to water all the flowers.

Which of the flower beds (A-D) will not be watered correctly?

A) 

B) 

C) 

D) 

Solution

The correct answer is C.

This drone has a problem. In instruction 1, when above a «bloomed flower,» it «moves right.» Then, in instruction 3, it also «moves right.» This means the drone may move two cells to the right when above a bloomed flower.

Let's trace the drone's movement for choice C:

- 1st cell (wilted) → water → move right
- 2nd cell (bloomed) → move right → move right again (2 cells!)
- 4th cell (wilted) → water → move right
- 5th cell (bloomed) → move right → stop

The 3rd wilted flower is skipped, and watering fails.

In choices A, B, and D, all wilted flowers can be watered.

This is Computer Science!

The drone in this task operates according to an algorithm. An algorithm is a sequence of instructions arranged in order to solve a task. Computers and robots operate according to algorithms created by humans.

However, if there is a mistake (bug) in the algorithm, it may not work as expected. Finding and fixing such bugs is called debugging. The drone's algorithm in this task had a bug that causes it to skip one cell under certain conditions.

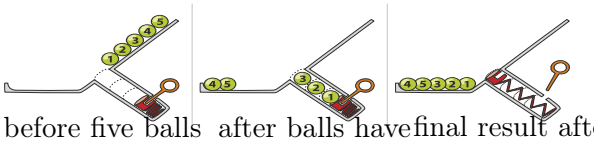
Programmers test with various input patterns to check if their algorithms have bugs. Such an input pattern used to reveal a bug is called a test case. In this task, finding which flower bed pattern causes the bug is exactly this kind of testing.

Websites And Keywords

TODO

28 Slope

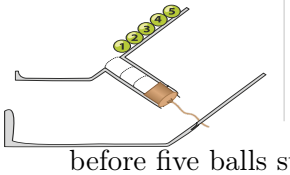
Numbered balls roll down ramps. When a ball comes to a hole, if the hole is not full, the ball falls in. Otherwise, the ball rolls past the hole. There are 2 kinds of hole stoppers. A stopper-pin  at the side of each hole can be pulled which ejects the balls. The cork  at the bottom can be pulled and just pass the ball to the tray below. Here is an example:



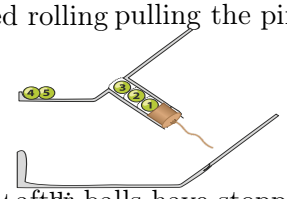
before five balls start rolling

after balls have stopped rolling

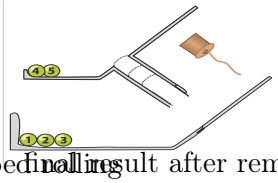
final result after pulling the pin



before five balls start rolling

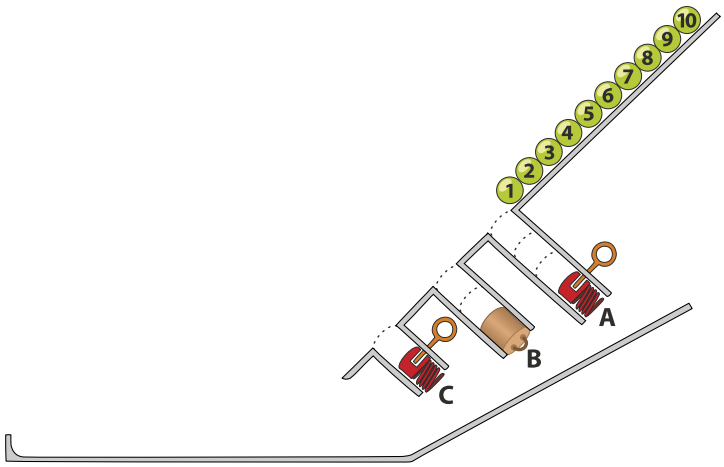


after balls have stopped rolling



final result after removing the cork

Ten balls roll down the ramp below. Three holes A, B and C have space for 3, 2 and 1 balls as shown. The holes A and C have a stopper-pin at the end and the hole B has a cork. Pins and/or corks are pulled in the order A, B, C but each time, only after all balls have stopped rolling.



Which option shows the final result?

A) 

B) 

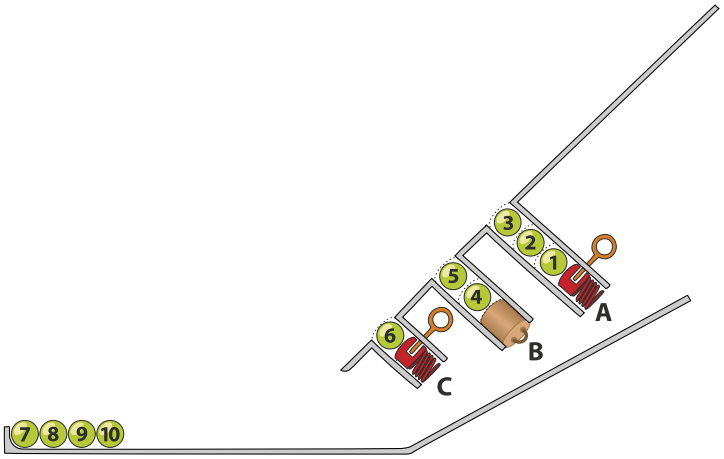
B) 

D) 

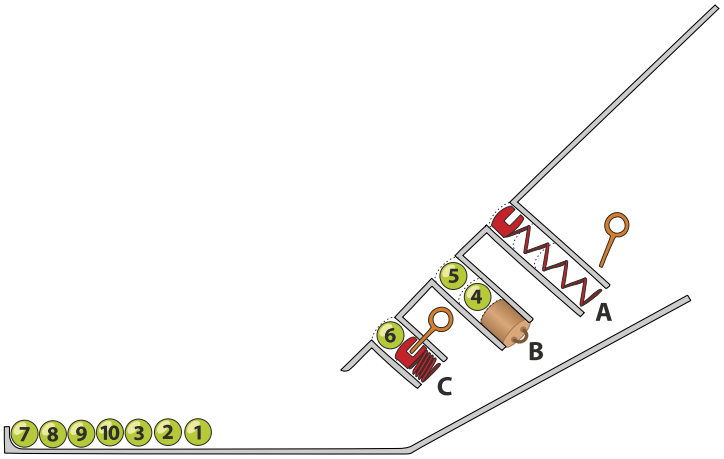
Solution

The correct answer is D.

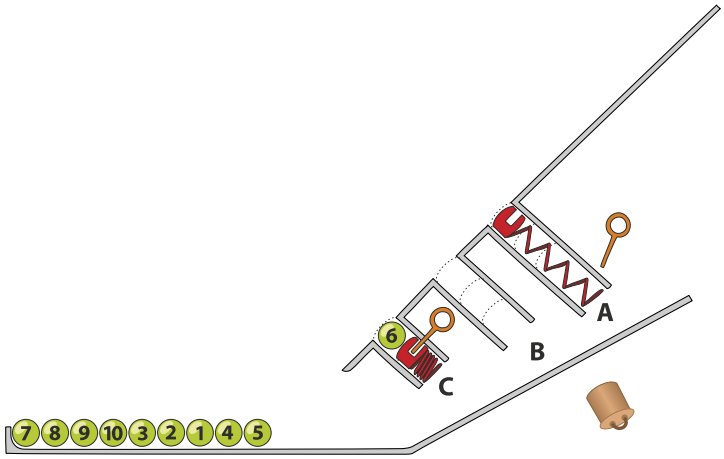
Hole A has space for three balls, so balls 4 to 10 roll past it in order. Hole B has space for two balls, so balls 6 to 10 roll past it in order. Hole C has space for one ball, so balls 7 to 10 roll past it in order.



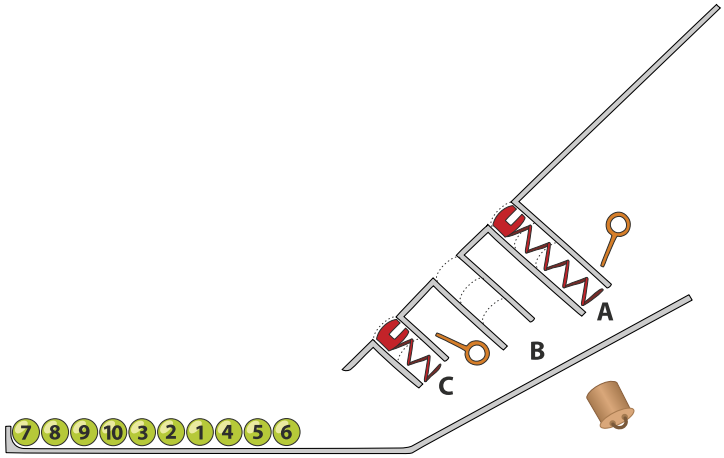
Then the pin in hole A is pulled and balls are ejected in the order 3,2,1 and roll to the bottom. At this point, the balls are at the bottom in the order 7,8,9,10,3,2,1.



Then the cork in hole B is pulled and balls are passed in the order 4,5. At this point, the balls are at the bottom in the order 7,8,9,10,3,2,1,4,5.



Finally, the pin is pulled in hole C and ball 6 rolls to the bottom giving the correct answer.



This is Computer Science!

In computer science, sometimes we need to keep data in a certain order, and data structures can help us achieve this. Two important examples are stacks and queues.

When a hole is sealed with a stopper-pin, it acts like a stack data structure. The balls fall into the hole one by one. The first ball in (Ball 1) ends up at the very bottom, while the last ball in (Ball 3) stays at the top. When the pin is pulled, the balls are ejected back onto the ramp in reverse order. This is called LIFO (Last-In, First-Out). Think of a stack of cafeteria trays or the «Undo» button in a text editor—the last action you performed is the first one that gets removed.

When a hole uses a cork, it mimics a queue data structure. Instead of being ejected back the way they came, the balls are released through the bottom of the hole onto a path below. The first ball to enter the hole is the first one to land on the lower track. This is called FIFO (First-In, First-Out). Think of a line at a grocery store or a printer's document list—the first person or document in line is the first one served.

If a programmer needs to reverse a sequence of numbers, they would use a stack. If they need to process tasks in the exact order they arrived, they would use a queue.

Websites And Keywords

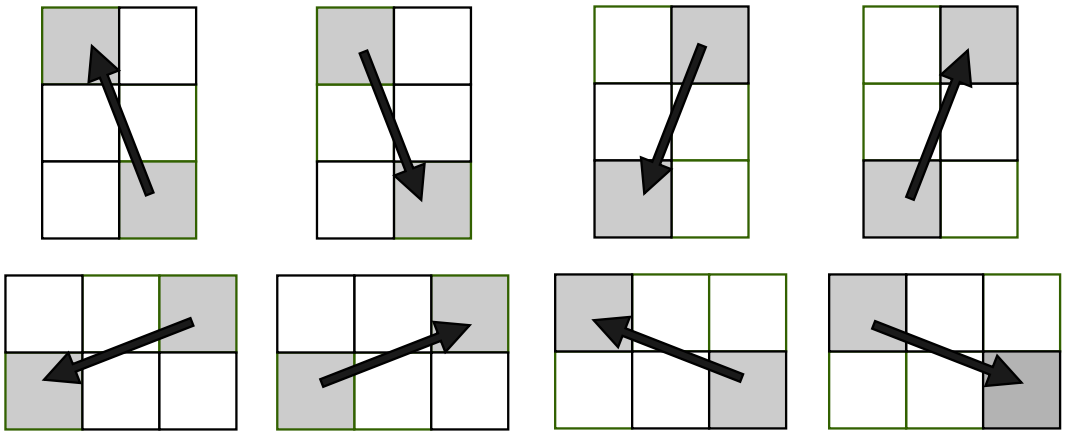
TODO

29 Jumping Horse

A horse is jumping through a field that is split up into 25 squares.

There are two rules:

- It can only visit each square once.
- It can only jump in the following manner:



The horse starts at the square marked 21 and wants to visit every single numbered square. Jumps into white squares are not allowed.

1		3		
	7			10
	12	13	14	
16		18		
21				25

In what order will the horse jump through the field?

A) START 21, 12, 3, 10, 13, 16, 7, 18, 25, 14, 1 FINISH

B) START 21, 12, 3, 14, 25, 7, 16, 13, 10, 18, 1 FINISH

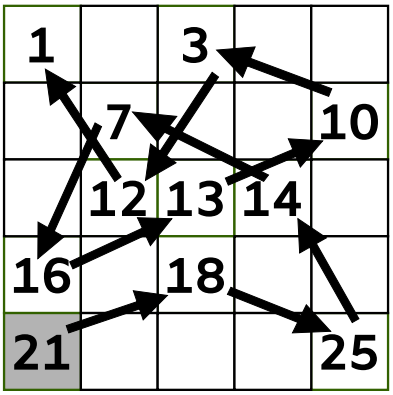
C) START 21, 18, 25, 14, 7, 16, 13, 10, 3, 12, 1 FINISH

D) START 21, 18, 7, 16, 13, 10, 3, 12, 1, 25, 14 FINISH

Solution

C is the correct option.

You can see the jumps taken by the horse below.



From the rules and the layout of the field we can deduct that field 1 can only be jumped on at the end, as there is no way to leave the square without jumping out the way you came. As from square 21 you have to options to jump to (12 and 18), it makes sense to start at 1.

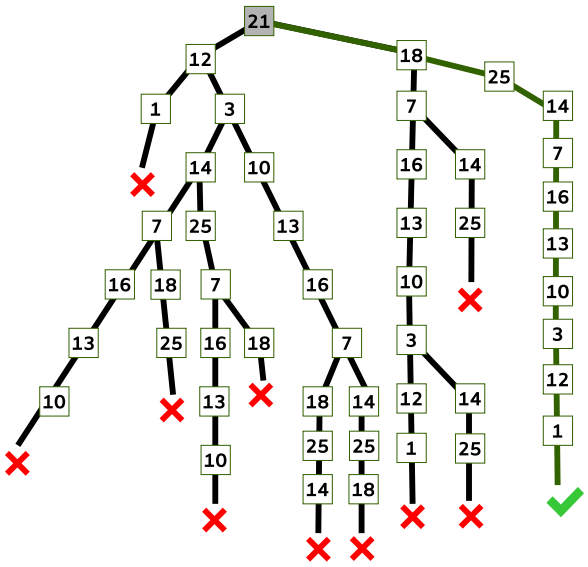
From there, we can deduct that 12 is the jump before the last, as it is the only square we can jump to.

From square 12 we can jump to 3 or 21. But as 21 is our starting point, it makes no sense to jump there. So tile 3 is our previous jumping point.

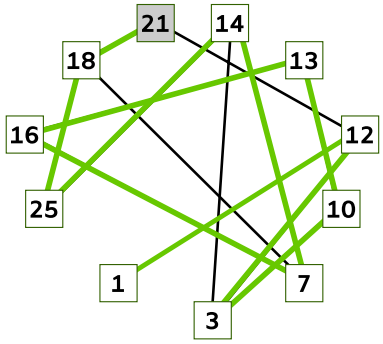
From square 3 we can reach either 10 or 14 and there are many options following it. However, if we go back to the actual start, 21, where we had the options 12 or 18, we can now exclude 12, as it has to come before the jump to 1.

From here, we can figure out the other jumps with more ease.

You can see all possible paths in this tree:



Another way to visualize the paths possible is to make a graph connecting the squares that can be reached. Each square is represented as a node, each line is a connection. The green line is the correct path. We see that square 1 only has one connection so this HAS to be the start or end point, since we start on the red square, it is the end point. We also see that only square 3, 7, 12, 14 and 18 have more than 2 connections so one line will remain unused.



This is Computer Science!

The Gray code is a special form of coding in which adjacent code words always differ in exactly one position. The difference between two code words is therefore always 1 (also known as the Hamming distance). This property remains constant between all consecutive code words. In addition, the code is cyclic, which means that the last and first code words also differ in only one position.

This property helps to reduce transmission errors, as it prevents incorrect intermediate states from occurring when several positions are changed simultaneously, for example due to signal interference or signal distortion. This is why Gray code is one of the most robust, error-avoiding coding methods.

Websites And Keywords

TODO

30 Beaver Juice Shop

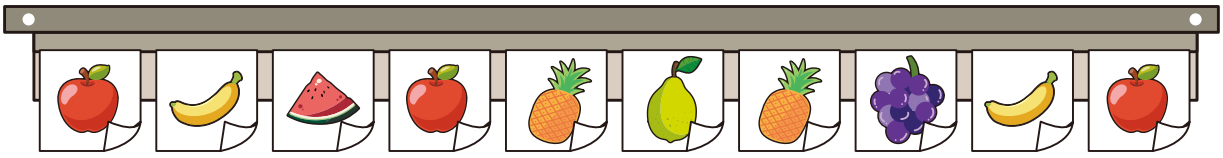
There are exactly three blenders in a beaver's juice shop.

The beaver can operate only one blender at a time and wants to avoid mixing flavors.

At the start of the day all three blenders are clean. Then the beaver satisfies requests following these rules:

- If a used blender matches the current request flavor: use it.
- Otherwise, if a clean blender is available: use it.
- If no matching or clean blender exists: clean the blender that has not been used for the longest time and use it.

Today's juice flavor requests in order (from left to right) are:



How many total cleanings are needed to complete all these requests?

A) 4

B) 5

C) 6

D) 9

Solution

The correct answer is (B) 5 times.

The table below shows the simulation of the usage while fulfilling the requests: The shaded cell marker marks the blender used for that request.

According to the rules, if there is a blender whose last-made flavor matches the current request, use it directly. If none matches, the blender that has been unused for the longest time must be cleaned cleaning before making the new flavor.

Request	Blender 1	Blender 2	Blender 3	Explanation
–				Start of the day.
apple	apple			Use blender 1.
banana	Unused	banana		Use blender 2.
watermelon	Unused	Unused	watermelon	Use blender 3.
apple	apple	Unused	Unused	Use blender 1.
pineapple	Unused	clean before pineapple	Unused	No blender matches. Blend
guava	Unused	Unused	clean before guava	No blender matches. Blend
pineapple	Unused	pineapple	Unused	Use blender 2.
grapes	clean before grapes	Unused	Unused	No blender matches. Blend
banana	Unused	Unused	clean before banana	No blender matches. Blend
apple	Unused	clean before apple	Unused	No blender matches. Blend

As a result, completing today's requests requires 5 cleanings.

This is Computer Science!

In computer systems, caches are small, fast storage spaces used to speed up program execution. They are faster than main memory (RAM) but have limited capacity. Caches store recently used or frequently accessed data so that future access is faster, reducing the need to repeatedly retrieve data from slower memory and improving overall performance. When a cache is full and new data must be stored, the system must decide which existing data to remove. This decision process is called a cache replacement policy. One commonly used policy is Least Recently Used (LRU). Under LRU, the data that has not been accessed for the longest time is replaced. This policy is based on the idea that data unused for a long period is less likely to be needed again soon.

This task is a contextual simulation of the LRU concept. The three blenders represent three cache slots, and each blender's last-made flavor represents the data currently stored in the cache. When a request matches a blender's flavor, this corresponds to a cache hit. When none matches, a cache miss

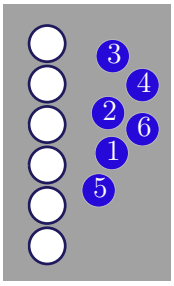
occurs, and a blender must be selected, cleaned, and updated according to the LRU rule. The rules to select the blender to be cleaned is a direct application of the LRU replacement strategy.

Similar mechanisms appear in daily life. For example, when working at a desk with limited space, people usually remove books that have not been used for a long time to make room for the materials currently needed. Likewise, the "recent files" list on a computer or cloud drive keeps recently opened files at the front and removes those not opened for a long time. This reflects the same principle: keep recently used items and replace those least recently used.

Websites And Keywords

TODO

31 Coins



There are 6 coins, labeled with the numbers 1 to 6. There are also 6 slots, which are likewise labeled 1 to 6.

A. At the beginning, the coins are placed randomly in the slots. The following procedure is then carried out to place all coins into their correct slots.

B. Consider the slots from top to bottom and take the first coin that is in the wrong slot. Place this coin to the right of the slot whose label matches the value of the coin.

This empties the slot from which the coin was taken.

C. Into this empty slot, place the coin whose number corresponds to that slot.

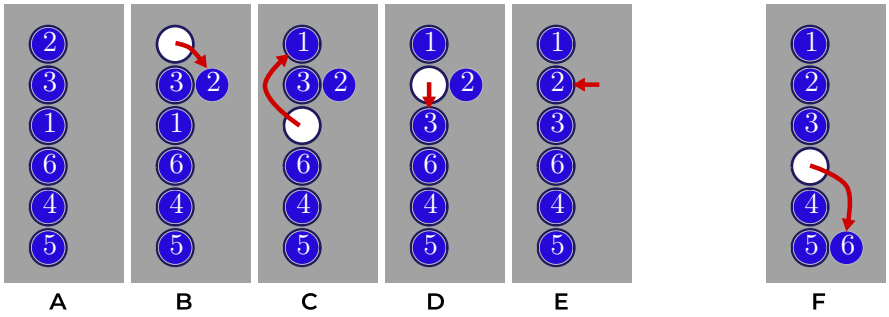
D. This makes another slot empty, which is then filled with the coin that belongs there.

This process continues until the slot next to the coin that was placed aside becomes empty.

E. The coin at the side is then placed into this slot, so that all slots are occupied again.

The entire sequence of these movements (A-E) is called one turn.

F. The procedure can then be repeated with another turn until all coins are in their correct slots

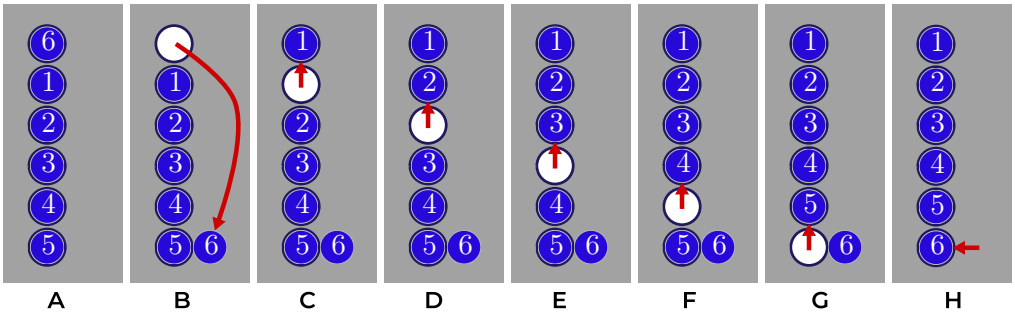


Assuming that initially no coin is in the correct slot: What is the minimum number of turns required to move every coin to the correct slot? What is the maximum number of turns that may be required for that?

A) 1, 6
B) 2, 3
C) 1, 3
D) 2, 6

Solution

The correct answer is: 3) 1, 3



The example above shows that a solution is possible in one turn.

Upper bound:

Let's define a cycle as a group of coins where each coin is sitting in the place of another coin from the same group, forming a loop.

For example, a cycle of 3 coins could be:

- Coin 1 is in slot 2
- Coin 2 is in slot 3
- Coin 3 is in slot 1

Each turn fixes one whole cycle.

Non-trivial cycles (i.e. cycles of misplaced coins) contain at least 2 elements, since a correctly placed coin forms a trivial cycle of length 1 and is not part of a cycle of misplacements

So with 6 coins, the largest possible number of cycles is:

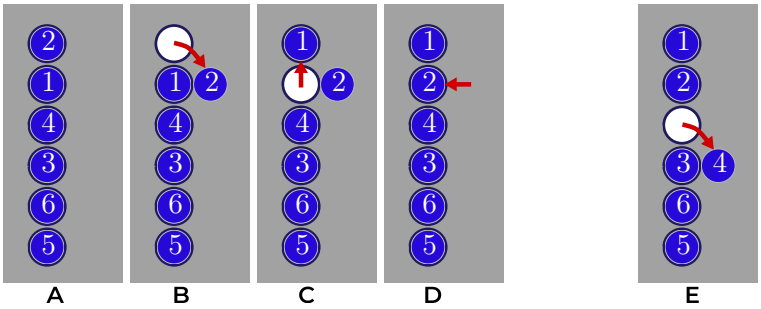
$6 \div 2 = 3$

For example:

$1 \to 2 \to 1, 3 \to 4 \to 3, 5 \to 6 \to 5$

Therefore, for 6 coins, at most 3 turns are required.

The image below shows the algorithm working on the sequence above.



This is Computer Science!

The procedure in this task is a sorting method. Sorting means arranging data so that everything ends up in the correct position. Computers do this very often, for example when ordering numbers, organizing files, or managing data in memory.

In this problem, each coin already indicates where it belongs: the number on the coin corresponds to the compartment with the same number. The goal is therefore to arrange the coins in the order 1, 2, 3, ... Sorting numbers into this fixed order is a special case of the general sorting problem, where the correct position of an element is not known beforehand and must be determined during the sorting process.

The method follows a clear algorithm: a precise sequence of steps that can be executed systematically until the correct arrangement is reached.

Another interesting aspect is that the method requires almost no additional memory. Apart from a single temporary place (the coin placed next to a compartment), all movements happen directly within the existing compartments. In computer science, such algorithms are called in-place algorithms, meaning they rearrange the data without needing extra storage.

How many turns are needed also reflects another important activity in computer science: analyzing algorithms. Computer scientists often investigate how many steps an algorithm needs in the best case and in the worst case.

Cost

The cost of the algorithm can be measured by counting the number of coin movements. Each coin is moved exactly once during the overall process, except that in each turn the first coin is moved twice (once when it is taken out and once when it is placed back).

If there are (n) coins and the process requires (m) turns, the total number of movements is therefore: $n + m$ In the worst case, each turn places only two coins into their correct slots, so at most $(n/2)$ turns are needed. This gives a maximum of $n + n / 2 = 3/2 \cdot n$ movements.

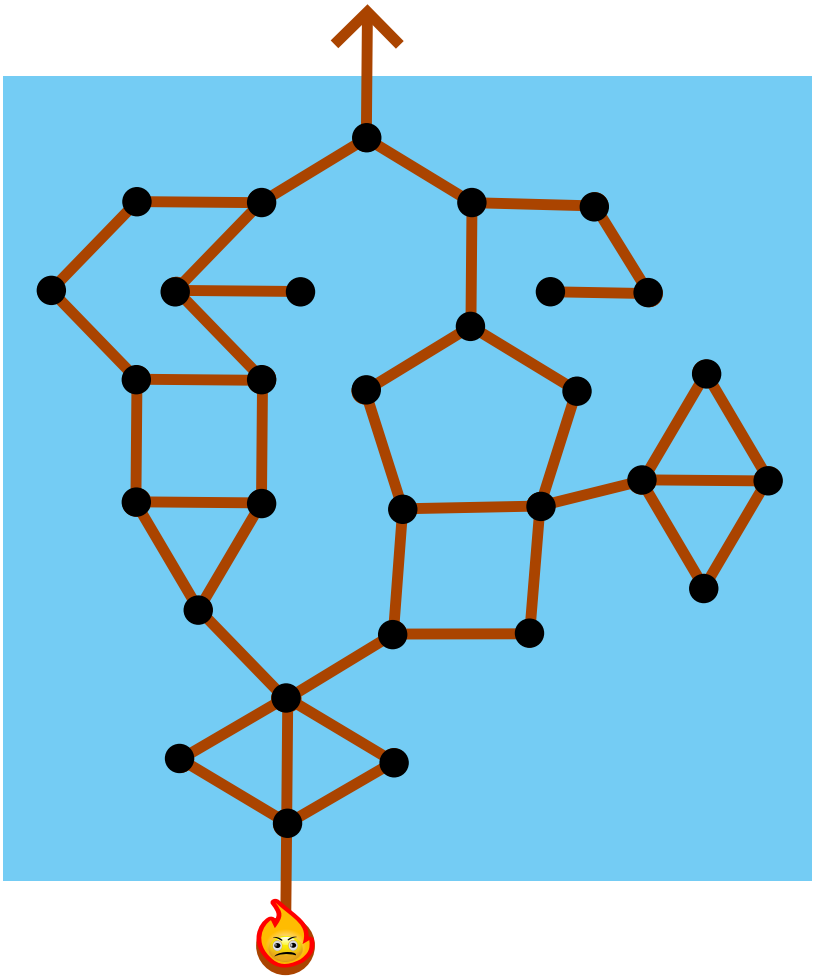
Since constant factors are ignored in Big-O notation, the cost of the algorithm is $O(n)$. (`https://en.wikipedia.org/wiki/Big_O_notation`).

Websites And Keywords

TODO

32 Fire Sprite

There are a number of islands on the lake, connected by footbridges.



The fire sprite (a fairy tale character) wants to get from where he is standing, up to the arrow. He wants to visit as many islands as possible. He cannot swim because the water would extinguish him.

However, the footbridges are wooden, and will burn down when the fire sprite walks on them. So the fire sprite cannot use any footbridge twice.

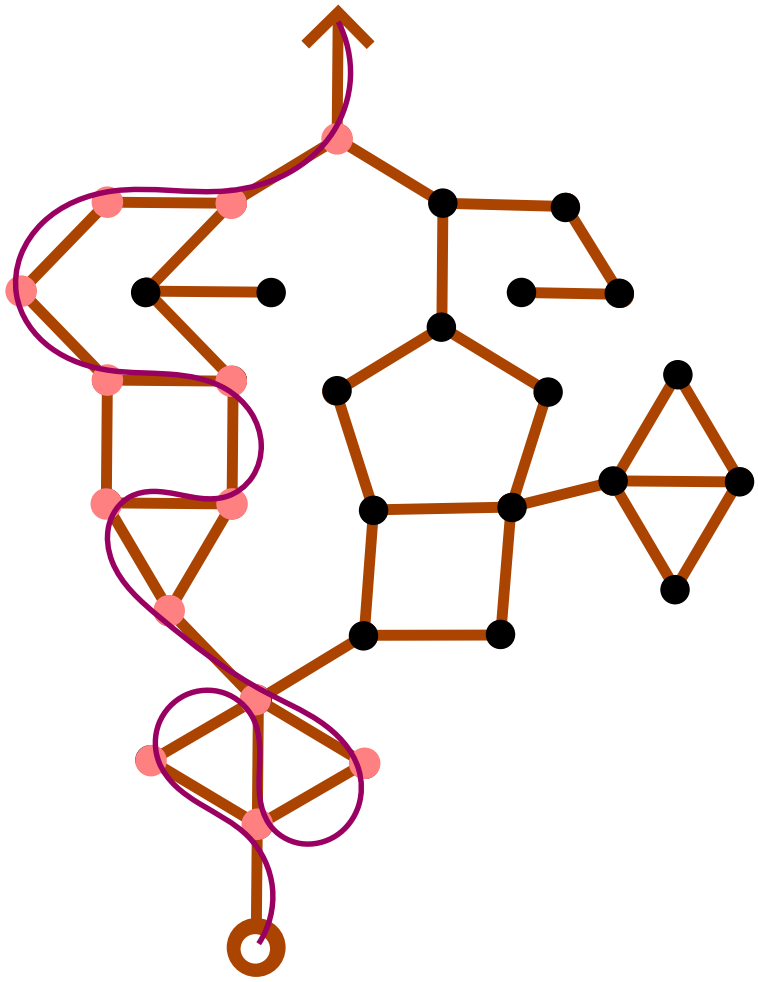
How many islands can the fire sprite visit at most?

A) 12
B) 13
C) 14
D) 15
E) 20

Solution

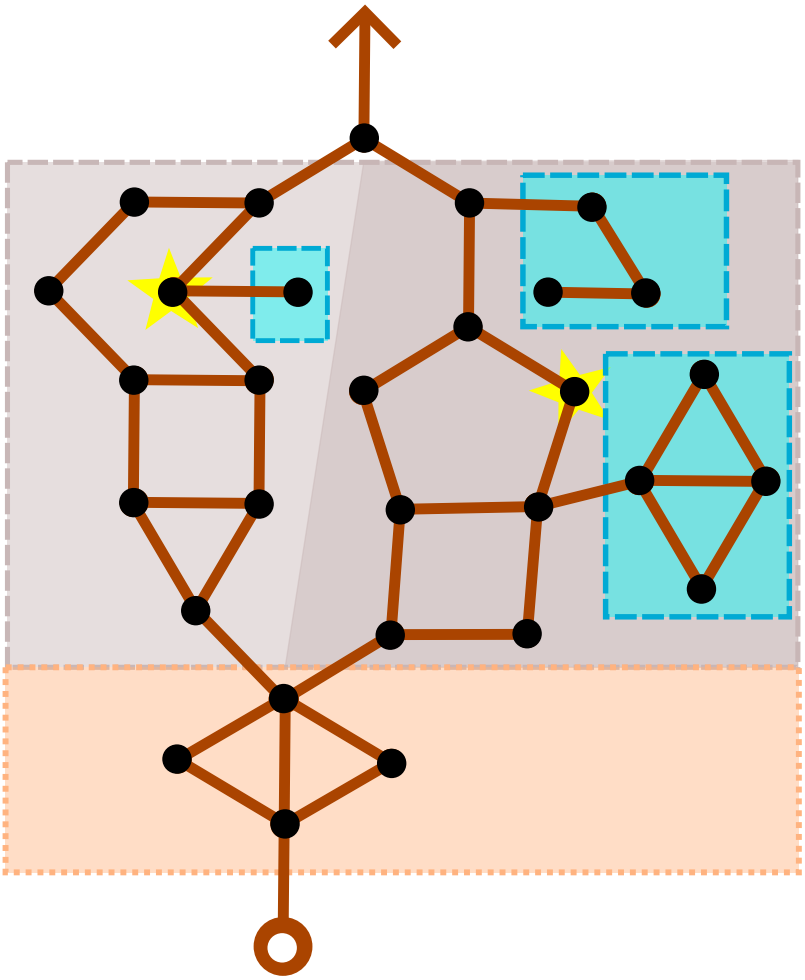
Correct answer is 13.

Correct solution is shown in the picture below. The optimal path is depicted by a curve and visited islands are highlighted.



How to get the correct answer:

We can divide the lake into two parts, the lower (beige dotted rectangle) and the upper (gray dashed rectangle), and solve these parts independently (lower picture).



We must also not forget about one island at the very end of the path.

The lower part contains enough bridges to pass through all 4 islands, while using each bridge at most once.

The upper part is divided into two branches, the left (light gray) and the right (dark gray). The fire sprite can only pass through one of these branches, because each of them has only one entrance and one exit footbridge.

In addition, there are groups of islands connected to the main part by only one bridge (small blue rectangles). The fire sprite cannot enter there, because he would not be able to return. We do not need to consider these parts.

We can compare these two branches. The left branch contains 9 islands, but it is not possible to pass through all of them. One island remains unvisited (the one marked with a star). It cannot be used otherwise more than one island would have to be missed. The right branch contains 8 islands and also 1 of them (marked with a star) cannot be used from the same reason. It is better to go through the left branch.

So choosing the left branch in the upper part leads to 8 islands taken. If we add 4 islands from the bottom and the 1 island at the very top, we get 13 islands.

This is Computer Science!

The system of connected islands on a lake in our task can be represented in computer science by a graph, i.e. a set of vertices connected by edges. Various graph tasks involve problems of how to traverse a graph. A typical task for traversing a graph is navigation to some location.

We can record a walk through a graph using a series of vertices and edges that are traversed. Some walks have restrictions, e.g. that each edge can be traversed at most once.

This task shows how graph theory can be used with robots. An example of such a task is planning a route for a robotic inspection of corridors or pipes without repeated sections and unnecessary returns, testing electrical networks to verify the functionality of all critical connections without repeated loading, a route for a security patrol or drone without returning through sections already traversed, or logistics routes inside warehouses without passing through narrow corridors that have already been traversed.

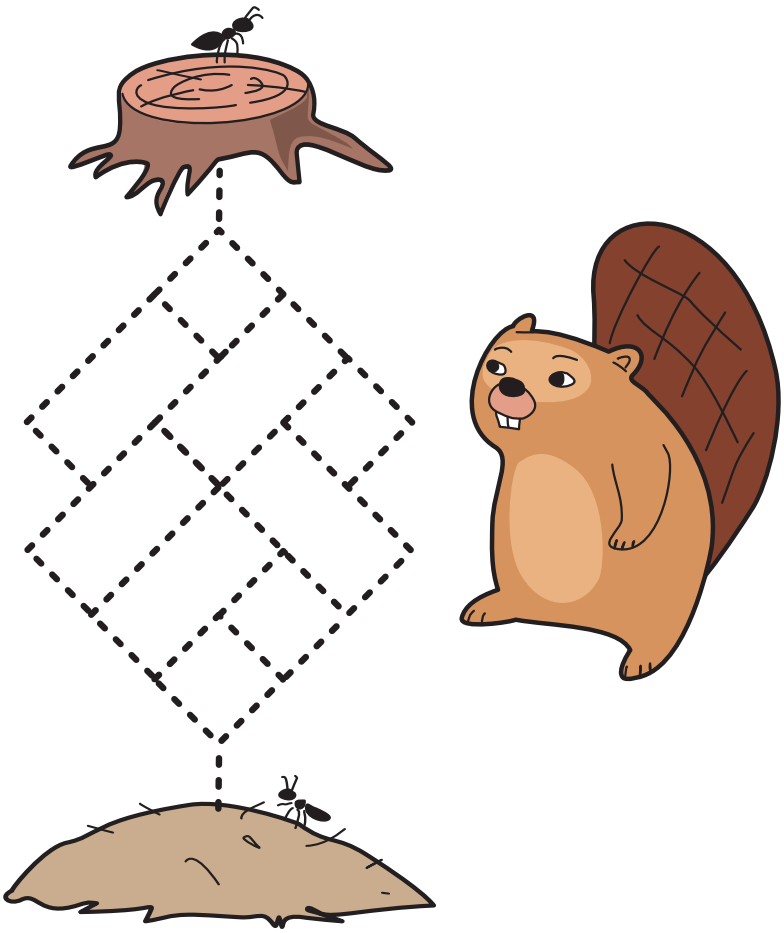
Websites And Keywords

TODO

33 The Anthill Puzzle

One day, a beaver was watching ants walking around and wondered how many different paths the ants take from the tree stump  to the anthill  .

The beaver drew a simple map, marking all possible paths with a dashed line. The ants want to get to the anthill as quickly as possible, so they never go back towards the tree stump.



How many different paths take the ants from the tree stump to the anthill?

A) 15

B) 16

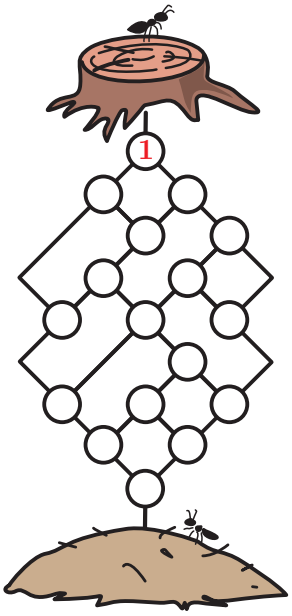
C) 17

D) 18

Solution

The correct answer for the task is 17.

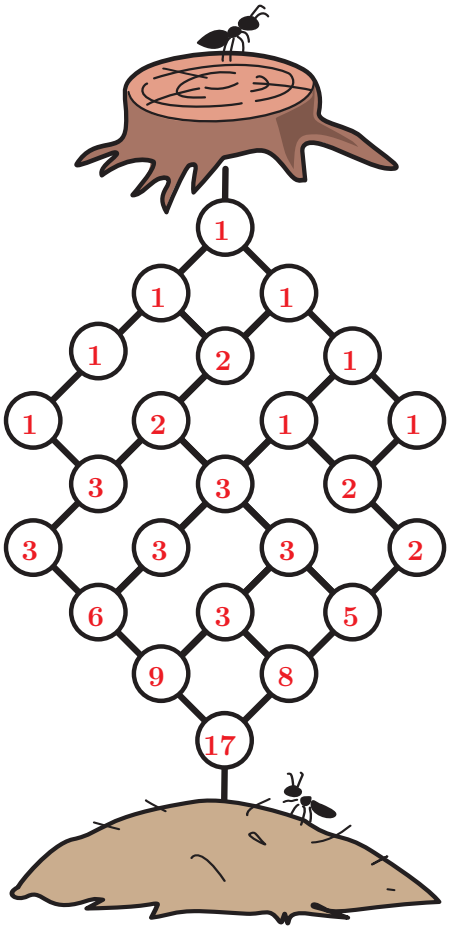
We can model the map with a graph, where we place a node in each intersection and an edge for each path.



Starting from the stump and moving downwards, we can label each node with the number of paths leading to it as follows:

- if the node only has one edge above, the number of paths leading to this node must be the same as the node above it, and hence, we label it with the same number.
- if the node has more than one edge coming from above, we need to look at the connected nodes. Remember that they are already labeled with all possible ways to reach them. Since all those paths lead to our node and they are all different, we have to label the node with the sum of their labels.

Using this method, we fill in the remaining nodes with numbers, top to bottom, as shown.



This is Computer Science!

This task is an example of a Computer Science problem because it uses several core ideas from algorithm design:

At first, it may look like a simple puzzle about ants choosing paths. But underneath, it is really about working with a network of connected points and counting how many ways there are to move from the start to the end.

The map can be seen as a directed graph. Each crossing point is like a node, and each path between two points is like an edge. Since the ants can only move downward, the direction of movement is fixed. This makes the map a special kind of graph called a Directed Acyclic Graph, or DAG. Counting all possible routes through this graph is a classic problem in informatics.

The task also uses dynamic programming. Instead of trying to list every possible ant path one by one, we calculate the number of ways to reach each point. For each crossing point, we add the number of paths coming from the points above it. This saves time because we reuse results we have already found instead of recalculating the same things again and again.

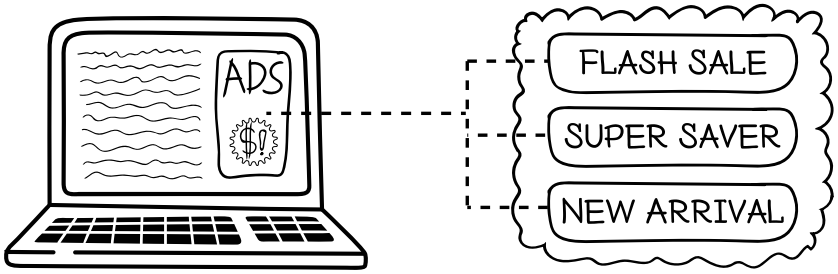
Finally, the solution relies on systematic counting. By working carefully row by row, we make sure that every possible path is counted exactly once. This helps avoid two common mistakes: missing some paths or counting the same path more than once.

Websites And Keywords

TODO

34 Online Ad

An online shop shows one advertisement each day on its webpage. There are three ads: Flash Sale, Super Saver and New Arrival. Each day the shop earns beaver coins from the ad shown. Over time, the online shop would like to earn as many beaver coins as possible.



The shop keeps a score for each ad. The score shows how good the ad has been so far.

The shop follows these rules to decide which ad to show:

- At the beginning all scores are zero.
- For the first three days, the shop shows one different ad each day, so that every ad is shown once.
- After an ad is shown on a day, its score is updated like this:

The new score of the ad is the average of its previous score and today's earnings. The scores for the other ads do not change.

- On every 3rd day after that (Days 6, 9, 12,...) the shop chooses the ad with the lowest score to explore alternatives.
- On all other days, the shop shows the ad with the highest score.
- If two or more ads have the same score, the shop chooses Flash Sale over Super Saver over New Arrival, in that order.

So far

- Day 1: earning = 20 beaver coins
- Day 2: earning = 20 beaver coins
- Day 3: earning = 80 beaver coins
- Day 4: earning = 20 beaver coins
- Day 5: earning = 20 beaver coins

Which ad will the system show on day 6?

A) Flash Sale
B) Super Saver
C) New Arrival
D) Cannot be decided

Solution

Correct Answer A.

Let us track the scores step by step.

Start: All scores are zero

After Day 1 (Flash Sale is shown due to tiebreaker because all scores are 0 and it earns 20): Flash Sale's score becomes the average of 0 and 20 = 10

After Day 2 (Super Saver is shown due to tiebreaker between the two unshown ads and it earns 20): Super Saver's score becomes the average of 0 and 20 = 10

After Day 3 (New Arrival is shown because it is the last remaining unshown ad and it earns 80): New Arrival's score becomes the average of 0 and 80= 40

Day 4 is not multiple of 3 so the shop chooses the ad with the highest score which is New Arrival.

After Day 4 (New Arrival earns 20): New Arrival's score becomes the average of 40 and 20 = 30

Day 5 is also not a multiple of 3 so again the shop chooses the ad with the highest score which is New Arrival.

After Day 5 (New Arrival sale earns 20): New Arrival's score becomes average of 30 and 20 = 25

Final scores at the end of day 5: Flash Sale:10, Super Saver:10, New Arrival:25.

Day 6 is multiple of 3, so the shop selects the ad with the lowest score. The lowest score is 10, which is a tie between Flash Sale and Super Saver. Using the tie breaking rule, the system chooses Flash Sale.

This is Computer Science!

This task models a classic reinforcement learning setting called the multi-armed bandit (MAB). In a multi-armed bandit problem, an agent repeatedly chooses one action from a fixed set (the "arms"), and after each choice, it receives a reward from that option, but how good the reward will be is not known in advance. The objective is to maximize total reward over many choices, while balancing learning (trying options) and earning (using the best-known option).

Concepts of exploration and exploitation are presented here. On normal days, the shop shows an ad that has rewarded the shop the most so far (exploitation). On special days (multiples of 3), the system selects the ad that has the worst reward so far (exploration). This is a trade off in decision making strategy – learning between searching for new possibilities versus maximizing returns. Balancing the two allows AI agents to maintain efficiency and stability.

Websites And Keywords

TODO

35 Beaver Tiles

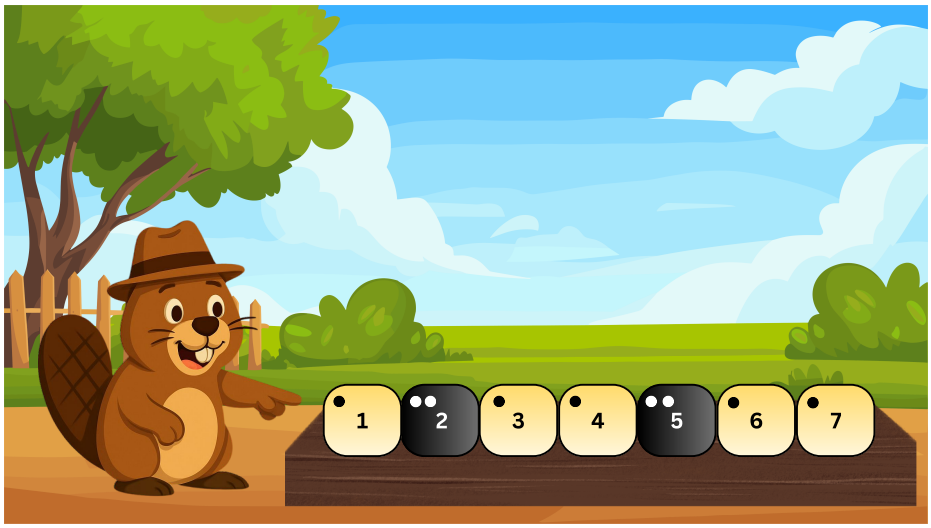
Beaver Kito has a row of 7 tiles. Each tile is Light on one side and Dark  on the other side.

In each move, Kito chooses exactly two tiles and flips both of them, so that:

- Light becomes Dark
- Dark becomes Light

Kito can repeat this move as many times as he wants.

The starting pattern of the tiles is shown in the picture.



Which of the following patterns is impossible to reach from the starting pattern?

A)

B)

C)

D)

Solution

The correct answer is B.

In each move, Kito flips exactly two tiles. This means:

- Two Light tiles can become Dark
- Two Dark tiles can become Light
- Or one Light and one Dark can swap

So, the total number of Dark tiles always changes by 0 or 2.

Because the change is always an even number, the parity of the Dark tiles (whether the number of Dark tiles is odd or even) never changes.

- Starting Pattern: There are 2 Dark tiles, which is an even number.
- Options A, C, and D: These have 4, 4, and 2 Dark tiles respectively. Since these are all even, they can be reached.
- Option B: This pattern has 3 Dark tiles, which is an odd number. It is impossible to reach an odd number from an even starting point.

This is Computer Science!

This task is about invariants. In informatics, an invariant is something that does not change, even after applying many operations.

Here, each move flips two tiles. Because of this, the number of Dark tiles can change by 0 or 2, but it can never change from even to odd, or odd to even.

Instead of trying all possible moves, we look for such unchanging properties. This helps us decide which situations are possible and which are not, without testing everything.

Invariants are widely used in informatics to:

- check whether algorithms work correctly,
- prove that certain states cannot be reached,
- reason about systems with many possible states.

Websites And Keywords

TODO

36 Tile Factory

A tile factory has two tile printing machines ( and ). To make a decorative tile, choose a stencil pattern and a tile and put both in one of the machines. The machine will then print the pattern on the tile. The two machines work a bit differently.

Pattern: 

Machine: 

Input Tile: 

How it Works: The machine repeats the following steps 4 times:

- print a stencil pattern on the top left quarter of the tile,
- then rotate the tile 90° clockwise.

Output Tile: 

Pattern: 

Machine: 

Input Tile: 

How it Works: The machine repeats the following steps 4 times:

- print a stencil pattern on the top triangle of the tile,
- then rotate the tile 90° clockwise.

Output Tile: 

A tile can be used as input tile more than once. When it is printed on more than once, a pattern printed later will be printed over all earlier patterns.

Which of the following sequences of machines and stencil patterns will produce this  *tile?*

A) 

B) 

C) 

D) 

Solution

The correct answer is option C.

To see this, you need to work backwards from the final decorative tile and check this for every pattern: if the pattern covers other patterns, it must have been printed after the ones it covers.

So first, identify which machine could have produced each visible pattern:

This is Computer Science!

In programming, modularization means organizing a program into reusable parts so that a complex process becomes easier to understand and maintain. One common form of modularization is a function. A function takes an input, follows a set of pre-defined rules and steps, and produces an output. This allows a long process to be reused and treated as a single action.

In this task, two printing machines can be viewed as two functions with different behaviors. Each machine takes a plain or partially printed tile as the input together with the chosen stencil as the parameter, and then based on its printing rules, produces a newly printed tile as the output. Even though each machine performs several internal steps, from the outside the user only needs to choose the machine and the stencil pattern. This input-output view makes the process easier to describe and analyze.

The task also illustrates sequential structure. A tile can be printed more than once, so the final tile is the result of applying several functions one after another. Sequential structure is one of the most fundamental ideas in programming. It means instructions are carried out in a fixed order, step by step. In many programs, changing the order of instructions often changes the result, even when the same instructions are used. Here, the printing order matters because later prints cover earlier ones. This concept connects to backward reasoning. Given the final output, we infer which operations happened earlier by looking at what is visible and what is covered. This is similar to debugging or analyzing a program's output to reconstruct which steps were executed and in what order.

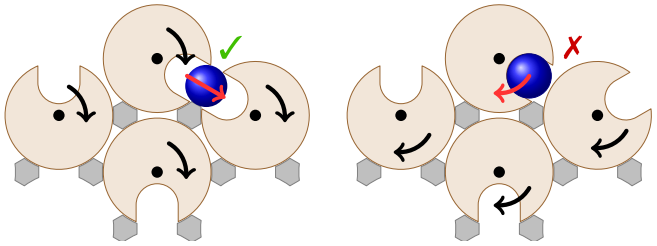
A similar idea appears in everyday life. For example, when ordering a drink, a customer chooses options like flavor, size, and ice level. The drink shop's system follows a fixed recipe to produce the drink. This is like a function: the options are the input, the finished drink is the output, and the customer does not need to know the step-by-step recipe inside. This shows why modularization is useful: complex work can be done through simple, clear inputs and outputs.

Websites And Keywords

TODO

37 Motorized Rollers

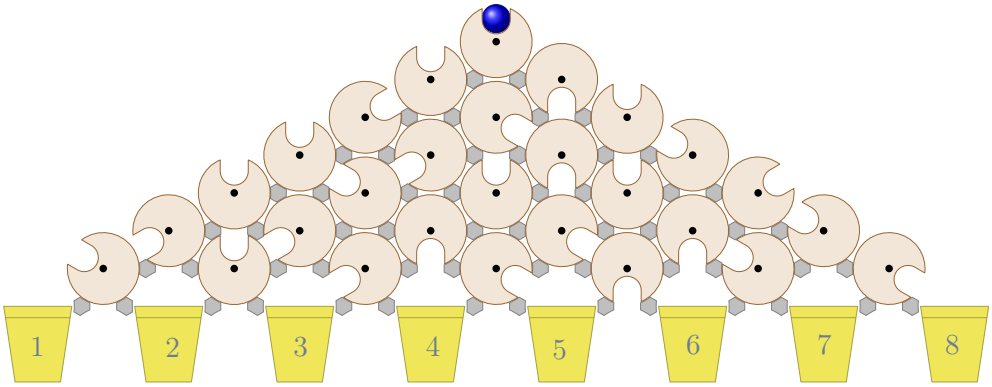
A ball is sent through an array of identical motorised rollers. The rollers slowly rotate in the same direction at the same steady rate. There is a switch that determines whether the rollers are all simultaneously rotating either clockwise or anticlockwise.



Each roller has a slot into which a ball can fit perfectly. When the slots of adjacent rollers line up with each other, the ball will drop from the higher roller into the lower one. (As in the left picture.) In the configuration on the right, the gaps will never align (given that the rollers all move in the same direction) and the ball will remain in the top roller.

As shown in the picture, plugs are placed between the rollers to ensure the ball can never drop into the gap between rollers or out of the array of the rollers.

A set of rollers is set up as below, with buckets placed to catch the ball.



How many of the buckets could the ball eventually reach?

A) 1

B) 2

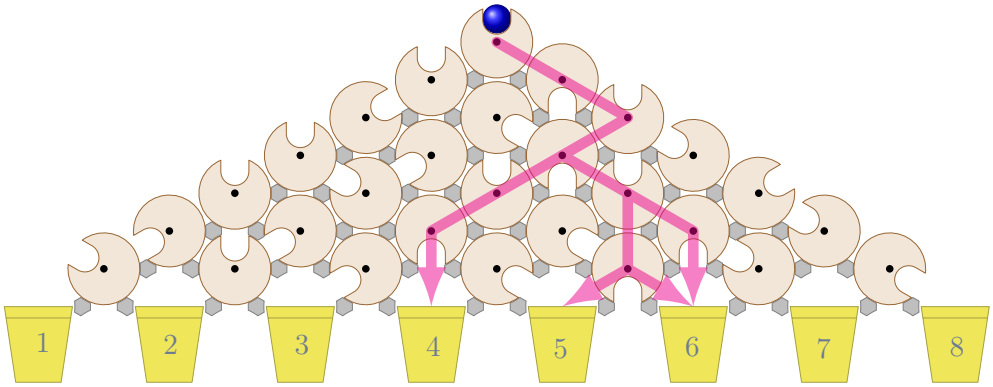
C) 3

D) 4

Solution

The answer is C) 3

The answer is buckets 4, 5 and 6, as shown in the diagram below which highlights the potential paths.



To identify the possible buckets, we can take a methodical approach, testing the potential alignment of the slots on the rollers.

For example, we can identify that if the top roller is rotated clockwise by 1/3, the rotation of the roller to the bottom right of it by the same amount in the same direction will align the slots, making it a valid transition for the ball. If we then check whether the top roller's slot will align with the slot on the bottom left roller, we can see that if we rotate the top roller 1/3 counterclockwise, the slot on the bottom left roller will be on the opposite side of the roller after the same rotation. Thus, the ball can never be transferred from the top roller to the bottom left and we can eliminate it from the tree, which also eliminates the furthest left bucket as an option.

From this assessment, it can be observed that two rollers will align only if the position of the slot on one roller is offset by 180 degrees from the slot on the other roller.

The algorithm works as follows:

- Start from the top roller.
- Examine all adjacent rollers.
- Check whether the 180-degree-alignment condition is satisfied.
- If the condition is satisfied, add the destination roller to the exploration queue.
- Repeat the process for all reachable rollers.
- Whenever a bucket is reached, record it as a possible destination.

This process can then be repeated for each subsequent roller to determine its possible destination roller

This is Computer Science!

The system of arrows used in the last picture is known in computer science as a directed acyclic graph, often shortened to DAG.

Directed means that the arrows point in one direction only — downwards, in this case. Acyclic means that no matter which path you follow along the arrows, you will always move downward and eventually reach the bottom; you can never return to an earlier point (there are no cyclics). A graph is a way of drawing connections between things, showing how they are related to one another — whether or not there is an arrow directly linking two items.

DAGs have many applications in computer science. For example, DAGs are used to plan tasks where some steps must be completed before others can begin, such as building software, processing data, or running calculations. They help computers (and people) understand what can happen in parallel and what must wait, making complex systems easier to organize, explain, and manage.

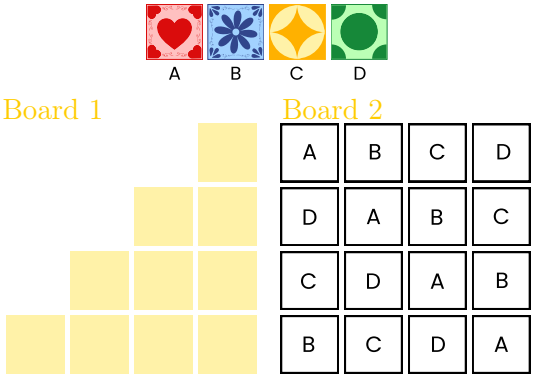
Once a proper DAG has been built, we determine all reachable buckets by following a graph-traversal algorithm, which will systematically explores all possible paths. For instance, the one used in the answer explanation is known as breadth-first search.

Websites And Keywords

TODO

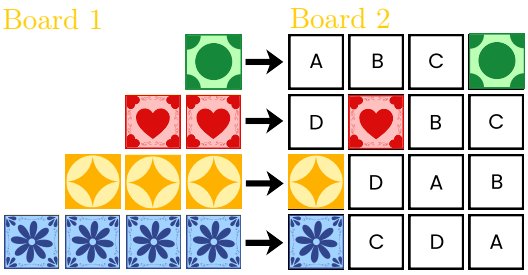
38 Azul Board Game

The beaver Laís is playing a board game. She has tiles of four different types (A, B, C, and D) and two boards (1 and 2). The letters on Board 2 indicate the type of tile that can be placed in each position.



The game consists of several rounds. In each round, Laís starts by placing tiles on Board 1. She can place as many tiles as she wants on Board 1 as long as each row contains the same type of tile, and each row is either complete or contains no tiles.

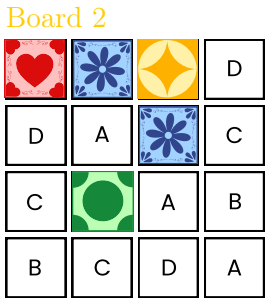
Then, for each completed row on Board 1, Laís takes one tile from that row and places it in the same row on Board 2, in the column corresponding to its type. The remaining tiles from that row on Board 1 are discarded.



This shows a possible result after the first round of the game.

The goal of the game is to complete the largest possible number of rows and columns on Board 2.

After few rounds of the game, Board 2 was:



In the next round, Laís wants to use at most 8 tiles. Which type of tile should she not use if she wants to complete the largest possible number of rows and columns on Board 2?

A)  B)  C)  D) 

Solution

Therefore, the correct answer is (B).

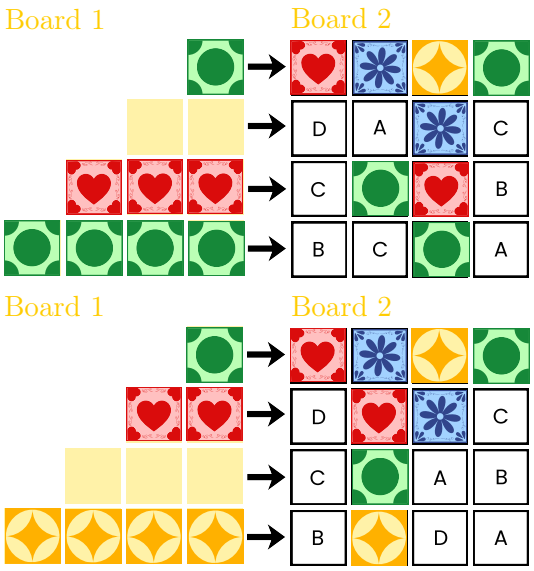
If Laís uses 8 tiles, then she can complete at most 3 rows in Board 1 if she completes the first row and any two other rows.

This allows Laís to complete the following rows and columns in Board 2:

- the first row, by placing a tile of type D;
- the second column, by placing a tile of type A in the second row and a tile of type C in the fourth row;
- the third column, by placing a tile of type A in the third row and a tile of type D in the fourth row.

However, the last two possibilities are mutually exclusive, because only one type of tile can be placed in the fourth row. Therefore, the best outcome in this round is to complete one row and one column on Board 2.

This leads to the following two options for Board 2. In each case, tile type B is not used.



This is Computer Science!

This task can be viewed as solving an optimization problem, as we want to maximize the number of completed rows and columns in Board 2. In computer science, an optimization problem is the problem of finding the best solution from all feasible solutions. Usually this involves finding the maximum or minimum value of something such as cost, distance, time, etc.

Websites And Keywords

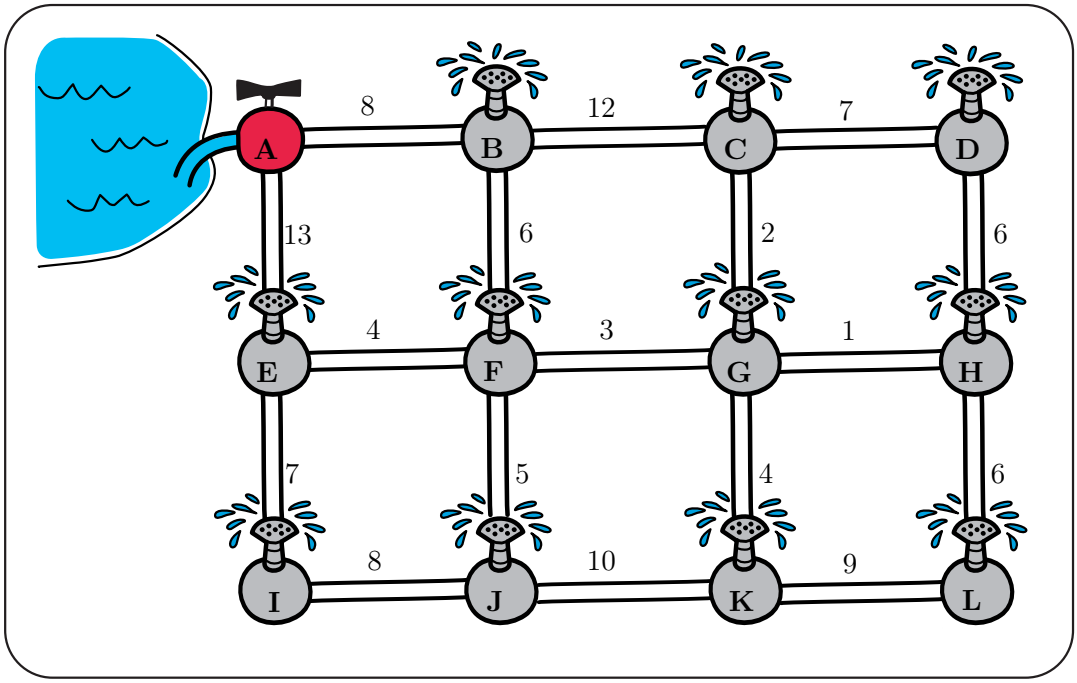
TODO

39 The Best Pipe Plan

Beaver Ben is installing a watering system in his field. Water starts from the main source A and must reach all sprinklers in the garden. Each sprinkler can be connected to at most three pipes.

Each pipe has a number indicating the cost of building it.

Ben wants to build the system as cheaply as possible, while still ensuring that water reaches every sprinkler.



What is the smallest total cost required to build such a network?

A) 57

B) 59

C) 60

D) 63

Solution

B is the correct option.

To build the cheapest pipe network, Ben should always consider the pipes with the smallest cost first.

First, choose the pipe with the smallest cost. If several pipes have the same smallest cost, any of them can be chosen.

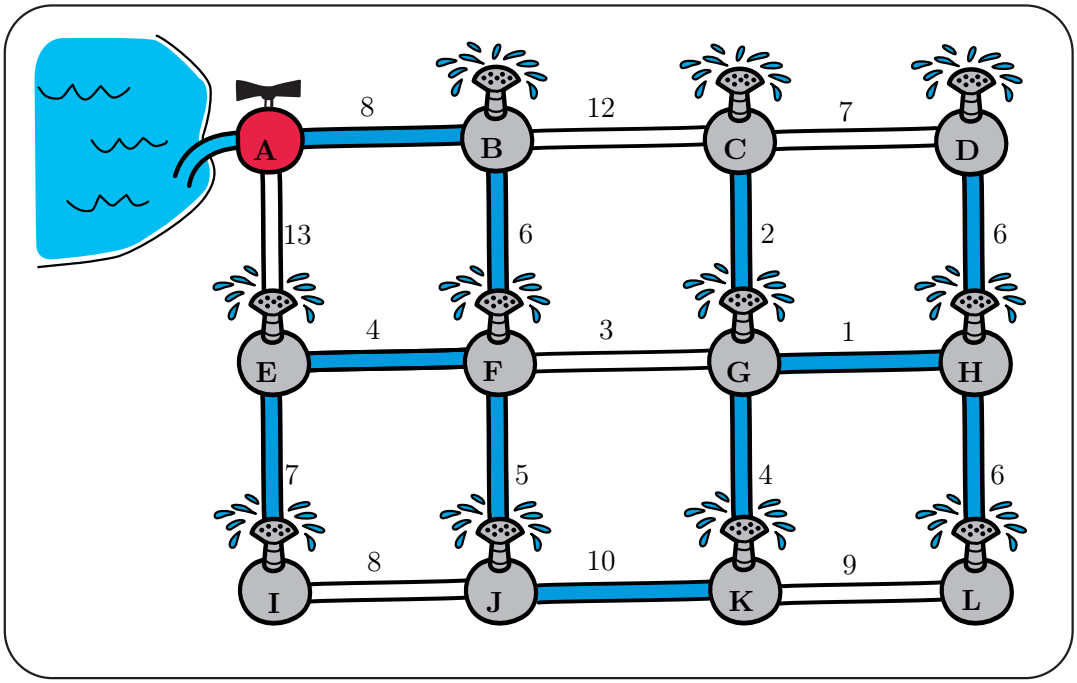
Then look at the next cheapest pipe. If adding this pipe would create a closed loop with the pipes already chosen, it should not be used because it would increase the total cost without helping to reach a new sprinkler. However, four pipes cannot be attached to one sprinkler.

Then we have to consider two cases: where the middle pipe FG is installed and where it is not.

1. If it is not installed, there are no four pipes attached to one sprinkler so we can continue choosing pipes in order of increasing cost and skip any pipe that would create a loop. In this way, new locations become connected step by step.

The process stops when all sprinkler heads are connected to the water source.

This method always produces the cheapest possible network that connects all locations. In informatics, such a network is called a Minimum Spanning Tree. The algorithm that finds it described above is known as Kruskal's algorithm.



2. If FG is installed, then one of the other pipes connecting F and one of the pipes connecting G should not be installed. For the whole system to be connected, we need at least 11 pipes, so at least 6 pipes should be in the outer edge, so it would cost at least $1 + 2 + 3 + 4 + 5 + 6 + 6 + 7 + 7 + 8 + 8 = 57$. If we would not take pipe BF, then we would need at least one of pipes of cost 12 and 13, so cost would increase by at least 4. Taking pipe BF increases

minimum cost by 1. If we do not take GK, we would need one of the pipes of cost 9 or 10, and would not need one of the pipes from top right square, so the cost would increase by at least 2. Taking pipe GK also increases cost by 2. So in this case the cost is at least 60.

This is Computer Science!

This task is about connecting several locations with the smallest possible total cost. In informatics, such problems are often represented as a network (graph) consisting of points and connections between them.

The goal is to connect all points while avoiding unnecessary connections and keeping the total cost as small as possible. The result is called a Minimum Spanning Tree. It connects all locations in the network without creating loops and with the smallest total cost.

Problems of this kind appear in many real situations. For example, when designing water pipe systems, electricity networks, road systems, or computer networks, engineers want to connect many locations while minimizing the cost of building the connections.

Computer scientists have developed algorithms that can automatically find such optimal networks. One example is Kruskal's algorithm, which gradually builds the network by choosing the cheapest connections while avoiding loops.

Websites And Keywords

TODO

40 Free Taco Order

Tacos are a unique part of Mexican culture: varied, delicious, and prepared fast!

A Taquería (food truck) promises:

«If your group order takes more than 15 minutes, the meal is FREE!»

This applies to orders of up to 8 tacos (any combination you want).

You and your friends want a free meal. The waiters assign the tacos upfront to the three cooks Ana, Beto and Carlos one by one, in the exact sequence in your order. They do it following these rules:

- Rule 1 (Fastest Cook): Give the new taco to the cook who prepares it the fastest.
- Rule 2 (Overload Protection): If that cook already has 10 minutes or more of work waiting, give the taco to next fastest cook instead.
- Rule 3 (Tie-Breaker): If cooks tie in speed, give it to the cook with the least amount of work. If there is still a tie, assign it in the next order: Ana, Beto, Carlos.
- Rule 4 (System Overload): If ALL cooks are overloaded (>= 10 mins), ignore Rule 2 and give the taco to the fastest cook.

Each cook has different speeds for each taco type.

See the table below of Preparation times in minutes:

Taco	Ana	Beto	Carlos
Paneer	3	5	5
Chicken	5	3	5
Veggie	2	11	11

Cooks prepare one taco at a time in the assigned order.

The total time stops when the last taco is finished.

Which of the following order sequences will successfully trick the Taquería rules and take more than 15 minutes?

A) 8 Veggie Tacos.

B) 8 Paneer Tacos.

C) 4 Paneer, 2 Chicken, and 2 Veggie Tacos.

D) 4 Chicken and 4 Veggie Tacos.

E) It is not possible

Solution

Option C is one correct answer:

4 Paneer, 2 Chicken, and 2 Veggie Tacos.

Let's trace Option C step-by-step as one successful way to hack the system:

1. 4 Paneer Tacos: Go to Ana (Fastest). She now has 4×3=12 minutes of work. (Ana is now overloaded ≥10!)
2. 2 Chicken Tacos: Go to Beto (Fastest). He has 2×3=6 minutes of work.
3. 1st Veggie Taco: The fastest is Ana, but she is overloaded. The waiter gives it to Carlos (least work between Beto and Carlos). Carlos takes 11 mins. (Carlos is now overloaded ≥10!)
4. 2nd Veggie Taco: Ana is overloaded. Carlos is overloaded. The waiter gives it to Beto. Beto had 6 minutes of work, plus 11 for the Veggie... Beto's total time is 17 minutes!

The Taquería Rules failed, and you win free food!

For other options:

Option A: 8 Veggies will end up with 5 V while Ana is first overloaded, and 1 V for Beto, 1V for Carlos, and then the last V goes to Ana again. Max time: 12mins

Option B: 8 Paneer will end up with 4 P while Ana is first overloaded, and 2P for Beto, 2P for Carlos. Max time: 12mins

Option D: 4 Chicken will first go to Beto, Beto is overloaded, and 4 Veggies will go to Ana. Max time: 12 mins

Option E: Since option C is a possible solution thus E is not correct.

This is Computer Science!

This task is about Adversarial Testing and Algorithm Tracing.

In computer science, a rigid set of instructions (like the Taquería rules) is called a Greedy Algorithm. It seems smart because it tries to use the fastest people and prevent them from getting too much work. However, rigid algorithms can be exploited.

Cybersecurity experts and software testers use «Adversarial Thinking»: they actively look for «Edge Cases» or "Worst Case Scenarios"—specific, tricky inputs (like this specific combination of tacos) that cause a system to fail or crash.

A better Load Balancing algorithm wouldn't just look at a rigid 10-minute limit. It would adapt dynamically to prevent disasters, like giving an 11-minute Veggie taco to a cook who is already busy, just because the specialist was slightly over the limit.

Websites And Keywords

TODO

41 Painting The Floor

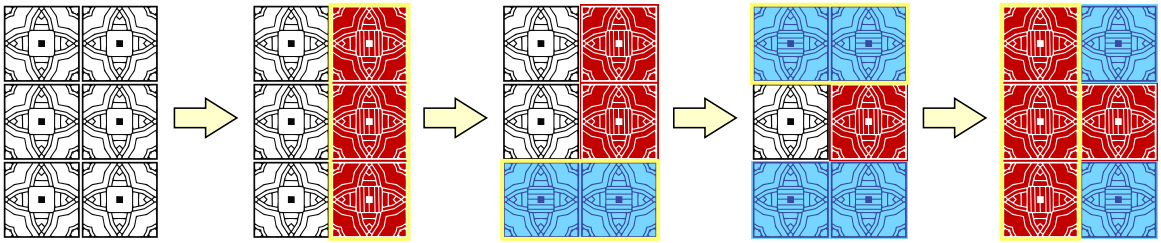
In order to prepare the room for the Bebras Convention, the organizers decided to paint the floor in blue and red.

The floor can be seen as a grid of initially colorless square tiles  . The organizers can only do two types of operations:

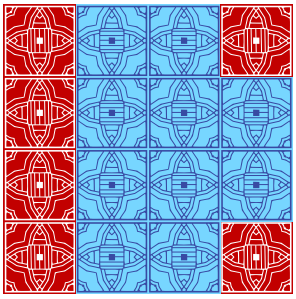
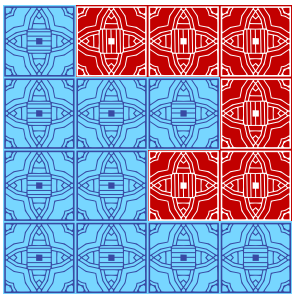
-  paint an entire row in blue
-  paint an entire column in red

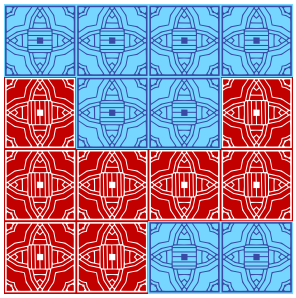
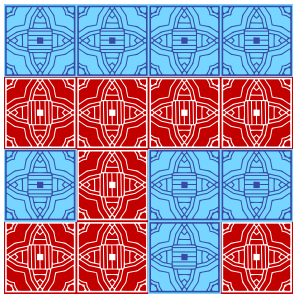
Tiles can be painted multiple times and their color will only depend on the last ink used.

For example, if the floor was a 3 rows by 2 columns grid, the following sequence of operations could be made:



Which of the following 4×4 colored floors cannot be obtained starting from an initially colorless grid?

A.  B. 

C.  D. 

Solution

TODO

This is Computer Science!

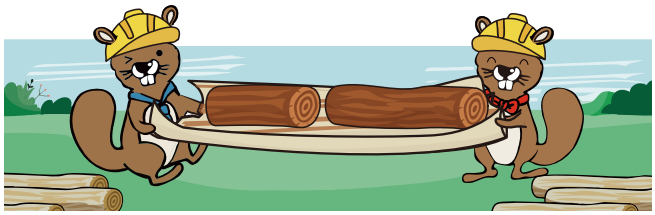
TODO

Websites And Keywords

TODO

42 Beaver Dam

Beaver engineers are building a dam and need a log that is 21 meters long.



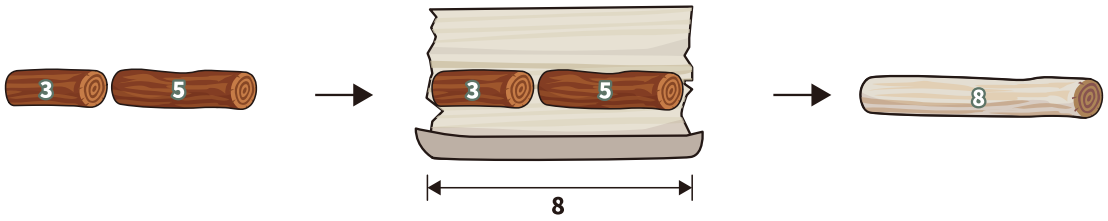
However, they only find six shorter logs which are 1, 2, 3, 4, 5, and 6 meters. They must join these six logs to form one log that is exactly 21 meters.

To keep the log strong, they follow these rules:

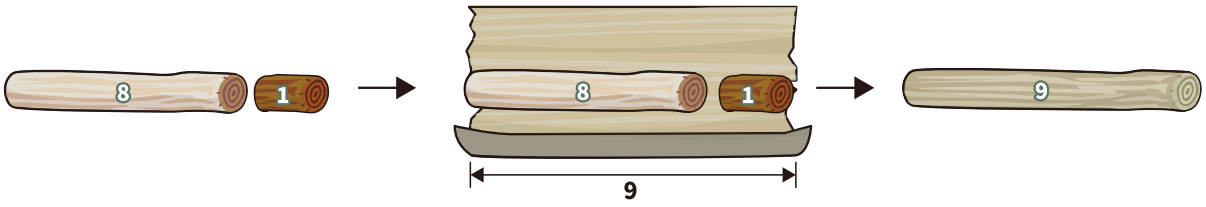
Each round, join exactly two logs end to end into one new log.

- Tightly wrap the entire new log with tree bark. The bark needed equals the length of the new log.
- The new log is treated as one log and can be joined with another log later.
- Take three logs with length 1, 3, and 5 meters for example:

If they join the 3-meter and 5-meter logs first, they need 8 meters of bark.



If they then join this 8-meter log with the 1-meter log, they need another 9 meters of bark.



When they join all six logs into one 21-meter log, what is the minimum total length of bark needed?

A) 45

B) 51

C) 56

D) 63

Solution

The correct answer is 51.

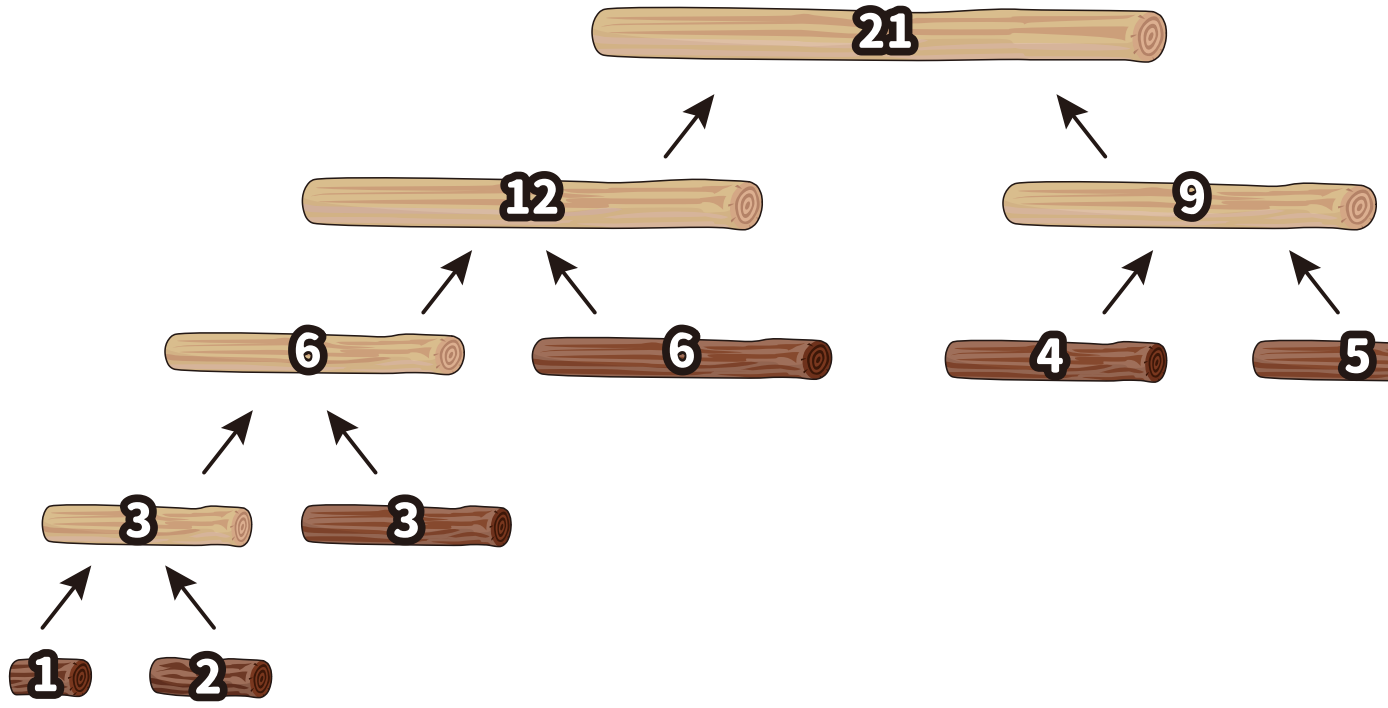
The table below shows a way to join all six logs into one 21-meter log in five steps using $3 + 6 + 9 + 12 + 21 = 51$ meters of bark.

Step	Available Logs (m)	Logs Joined	New Log (m)	Bark Needed (m)
1	1, 2, 3, 4, 5, 6	1 and 2	$1 + 2 = 3$	3
2	3, 3, 4, 5, 6	3 and 3	$3 + 3 = 6$	6
3	4, 5, 6, 6	4 and 5	$4 + 5 = 9$	9
4	6, 6, 9	6 and 6	$6 + 6 = 12$	12
5	9, 12	9 and 12	$9 + 12 = 21$	21

The idea behind this process is based on these observations:

- Each join creates a new log, and the bark needed for that join equals the new log's length.
- At each step, two logs are joined to become one, so with 6 original logs there will be 5 total steps. The total bark needed equals the sum of the 5 new-log lengths.
- If a newly formed log is joined again later, its length will be counted again in a future join. So, logs created early are more likely to affect the total more times.

This leads to a simple rule: always join the two shortest available logs. Notice how the choices in the table above follow this rule. We can view the process above as shown below. (The picture is called a tree.)



We can now more easily see how many times each log is wrapped in bark. For example, the original log of length 1 is wrapped in bark 4 times and the original log of length 4 is wrapped in bark 2 times. In general, the logs at level k (counting from 0 at the top) are wrapped k times. Considering each original log, we confirm that the total length of bark needed is 4 x (1 + 2) + 3 x 3 + 2 x (6 + 4 + 5) = 12 + 9 + 30 = 51.

As our problem's total cost function is identical to Huffman coding and optimal merge pattern problems, we know that the shortest pair first rule is optimal in those identical domains so it is also optimal in ours. We summarize the argument by exchange here (see the huffman coding reference later for formal proof that uses the argument by exchange with induction to cover all merge tree cases).

- Suppose we found an optimal result, R, that has a merge tree with total cost less than our rule would produce.
- X and Y are logs at the two maximum depth nodes in R's merge tree that did not follow the rule shortest pair first. length of X <= Y
- Because X and Y do not follow our rule, there must exist a log Z at a higher depth in the merge tree than Y but with shorter length
- If we swap log Z with log Y and then compute total cost of the new merge tree - old merge tree we get: change in total cost = ((depth of Y * length of Z) + (depth of Z * length of Y)) - ((depth of Y * length of Y) + (depth of Z * length of Z))

Refactoring, change in total cost = depth of Y * (length of Z - length of Y) + depth of Z * (length of Y - length of Z)

- Because length of Z < length of Y and depth of Y > depth of Z, the change in total cost will always be negative

This indicates swapping log Z with log Y would decrease the total cost. Thus our supposed optimal result R was not optimal.

Therefore, no merge tree ordering for this task can exist that has total bark required less than 51 metres.

This is Computer Science!

A Greedy Algorithm solves a problem by making the best choice at each step based on the current situation. The idea is that a sequence of locally optimal decisions may lead to a globally optimal result. Greedy strategies do not always guarantee the optimal solution. However, in this task, choosing the locally optimal option at every step leads to the global optimum. In this task, always merging the two shortest logs represents a greedy choice. This approach keeps intermediate logs as short as possible and reduces their repeated contribution to the total cost.

This task is closely related to Huffman coding, a greedy method for building an optimal prefix code. Each symbol is given a weight, usually its frequency or probability. At each step, the two smallest weights are merged. Repeating this builds a Huffman tree. Symbols deeper in the tree get longer bit

codes, and symbols closer to the root get shorter codes. Huffman's key result is that this process produces the minimum average code length among all prefix codes for the given weights.

Many real-world compression formats use this principle. For example, ZIP compression counts symbol frequencies and assigns shorter bit codes to frequent symbols and longer codes to rare ones.

Websites And Keywords

TODO